

LAPORAN KERJA PRAKTIK

PERANCANGAN DAN PENGEMBANGAN PLATFORM ASESMEN PSIKOLOGI DIGITAL UNTUK PUSAT KESEHATAN PSIKOLOGI (CHP) UNIVERSITAS KRISTEN KRIDA WACANA

Ditulis untuk memenuhi sebagian persyaratan akademik
guna memperoleh gelar Sarjana Ilmu Komputer Strata Satu



Oleh:

SANDERS KEANE DYLAN DANIYOLLA
412023020

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI CERDAS
UNIVERSITAS KRISTEN KRIDA WACANA
JAKARTA
2026



UNIVERSITAS KRISTEN KRIDA WACANA
FAKULTAS TEKNOLOGI CERDAS

PERSETUJUAN DOSEN PEMBIMBING KERJA PRAKTIK

**PERANCANGAN DAN PENGEMBANGAN PLATFORM ASESMEN
PSIKOLOGI DIGITAL UNTUK PUSAT KESEHATAN PSIKOLOGI
(CHP) UNIVERSITAS KRISTEN KRIDA WACANA**

Oleh:

Sanders Keane Dylan Daniyolla
(412023020)
Program Studi Informatika

Telah diperiksa dan disetujui untuk diajukan dan dipertahankan dalam sidang kerja praktik sebagai salah satu syarat yang diperlukan untuk memperoleh gelar Sarjana pada Program Studi Informatika, Fakultas Teknologi Cerdas.

Jakarta, 30 Juni 2026

Menyetujui:

Pembimbing

(Rita Wiryasaputra, S.T., M.Cs., Ph. D.)

Ketua Program Studi

(Dr. Gisela Nina Sevani, S.Kom., M.Si., M.M.)



UNIVERSITAS KRISTEN KRIDA WACANA
FAKULTAS TEKNOLOGI CERDAS

LEMBAR PENGESAHAN SIDANG KERJA PRAKTIK

**PERANCANGAN DAN PENGEMBANGAN PLATFORM ASESMEN
PSIKOLOGI DIGITAL UNTUK PUSAT KESEHATAN PSIKOLOGI
(CHP) UNIVERSITAS KRISTEN KRIDA WACANA**

Oleh:

Sanders Keane Dylan Daniyolla
(412023020)

Telah berhasil dipertahankan di hadapan Penguji dan diterima sebagai salah satu syarat yang diperlukan untuk memperoleh gelar Sarjana pada Program Studi Informatika, Fakultas Teknologi Cerdas.

Universitas Kristen Krida Wacana

Hari/Tanggal :

Tim Penguji :

Penguji I : Nama dan gelar Penguji I : _____

Penguji II : Nama dan gelar Penguji II : _____

KATA PENGANTAR

Puji dan syukur penulis panjatkan kepada Tuhan Yang Maha Esa atas berkat dan rahmat-Nya sehingga penulis dapat menyelesaikan laporan Kerja Praktik yang berjudul “Perancangan dan Pengembangan Platform Asesmen Psikologi Digital untuk Pusat Kesehatan Psikologi (CHP) Universitas Kristen Krida Wacana” dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu syarat untuk menyelesaikan mata kuliah Kerja Praktik pada Program Studi Informatika, Fakultas Teknologi Cerdas, Universitas Kristen Krida Wacana. Kerja Praktik ini dilaksanakan menggunakan skema Proyek Dosen di bawah bimbingan langsung Ibu Astin selaku Ketua Program Studi Psikologi dan Kepala Riset *Center for Health Psychology* (CHP) UKRIDA.

Penulis menyadari bahwa laporan ini tidak dapat diselesaikan tanpa bantuan dan dukungan dari berbagai pihak. Oleh karena itu, penulis ingin menyampaikan ucapan terima kasih yang sebesar-besarnya kepada:

1. Ibu Astin selaku pemilik proyek dan Kepala Riset CHP UKRIDA, yang telah memberikan kesempatan, arahan, dan umpan balik yang berharga selama pelaksanaan Kerja Praktik.
2. Dosen Pembimbing Kerja Praktik yang telah membimbing penulis dalam penyusunan laporan ini.
3. Seluruh dosen dan staf Program Studi Informatika, FTC, UKRIDA yang telah memberikan bekal ilmu dan dukungan selama perkuliahan.

4. Rekan satu tim yang telah berkolaborasi dalam pengembangan Platform Asesmen Digital CHP.
5. Keluarga tercinta yang senantiasa memberikan doa, motivasi, dan dukungan moral selama proses pelaksanaan Kerja Praktik.

Penulis menyadari bahwa laporan ini masih jauh dari kesempurnaan. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat bagi pembaca dan menjadi kontribusi yang berarti bagi pengembangan platform asesmen psikologi digital di Indonesia.

Jakarta, 30 Juni 2026

Sanders Keane Dylan Daniyolla

DAFTAR ISI

KATA PENGANTAR	iv
DAFTAR ISI	vi
DAFTAR TABEL	ix
DAFTAR GAMBAR	x
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Skema Kerja Praktik	8
1.3 Metodologi Pengembangan Berbantuan Kecerdasan Buatan . . .	9
1.4 Tujuan dan Manfaat	11
1.5 Batasan Kerja Praktik	14
1.6 Waktu Pelaksanaan	18
2 LANDASAN TEORI	20
2.1 Rekayasa Perangkat Lunak	20
2.1.1 Model Proses: Metodologi Agile	21
2.1.2 Pengembangan Perangkat Lunak Berbantuan Kecerdasan Buatan	22
2.2 Arsitektur Sistem Web Modern	23
2.2.1 Arsitektur Tiga Lapisan	23
2.2.2 Next.js dan <i>React Server Components</i>	24
2.3 TypeScript dan Keamanan Tipe	26
2.3.1 Signifikansi TypeScript dalam Proyek CHP	26
2.4 Backend dan Basis Data	27
2.4.1 PostgreSQL dan Basis Data <i>Serverless</i>	27
2.4.2 Drizzle ORM dan Perancangan Skema	27
2.4.3 tRPC	28
2.5 Autentikasi dan Otorisasi	29
2.5.1 <i>JSON Web Token (JWT)</i>	30
2.5.2 <i>Role-Based Access Control (RBAC)</i>	30
2.6 Pengujian Perangkat Lunak	32
2.6.1 Metodologi Pengujian	32
2.6.2 Strategi Piramida Pengujian	33
2.6.3 Pengujian Prosedur (<i>Mock DB</i>)	35
2.6.4 Pengujian Validasi: Akurasi Skor Otomatis	35
2.6.5 <i>Unit Testing</i> dengan Vitest	36
2.6.6 <i>End-to-End Testing</i> dengan Playwright	37
2.6.7 CI/CD dengan GitHub Actions	38
2.7 Keamanan dan Privasi Data	40
2.7.1 <i>OWASP Top Ten</i>	40
2.7.2 <i>Rate Limiting</i> dan Perlindungan API	41

2.7.3	Strategi Perlindungan Data Bertingkat	41
2.7.4	Model Tanggung Jawab Bersama pada Infrastruktur <i>Cloud</i>	42
2.7.5	Undang-Undang Perlindungan Data Pribadi	43
3	ANALISA DAN PERANCANGAN	45
3.1	Analisis Kebutuhan Fungsional	45
3.1.1	Kebutuhan Pengguna	45
3.1.2	Kebutuhan Perangkat Lunak	46
3.1.3	Kebutuhan Perangkat Keras	47
3.1.4	Diagram <i>Use Case</i>	48
3.2	Analisis Kebutuhan Non-Fungsional	50
3.3	Perancangan Arsitektur Sistem	52
3.3.1	Batas Klien- <i>Server</i>	53
3.4	Perancangan Basis Data	53
3.4.1	Model Domain	53
3.4.2	Skema Tabel	54
3.4.3	Relasi Antar-Entitas	58
3.4.4	Keputusan Perancangan Kritis	60
3.5	Perancangan Antarmuka Pemrograman Aplikasi	63
3.6	Perancangan Alur Antarmuka Pengguna	66
4	IMPLEMENTASI DAN PENGUJIAN	68
4.1	Lingkungan Pengembangan	68
4.2	Implementasi Basis Data	69
4.3	Implementasi <i>Scoring Engine</i>	70
4.3.1	Fungsi <i>reverseScore</i>	70
4.3.2	Fungsi <i>computeScore</i>	70
4.3.3	Fungsi <i>lookupInterpretation</i> (Lapisan Interpretasi)	72
4.3.4	Fungsi Validasi	73
4.3.5	Instrumen yang Diimplementasikan	73
4.4	Implementasi API <i>tRPC</i>	74
4.4.1	Komposisi <i>Router</i>	74
4.4.2	<i>Structural Lock</i>	74
4.4.3	Operasi Idempoten	75
4.5	Implementasi Autentikasi	75
4.5.1	Autentikasi Pengguna Publik (<i>Auth.js v5</i>)	75
4.5.2	Autentikasi Administrator (<i>jose JWT</i>)	76
4.5.3	<i>Rate Limiting</i> pada <i>Login Admin</i>	77
4.6	Implementasi Panel Administrasi	77
4.7	Implementasi Generasi PDF dan Pengiriman Surel	79
4.8	Pengujian Unit	79
4.9	Pengujian CI/CD	81
4.10	<i>Deployment</i> dan Opsi Instalasi Mandiri	82
4.10.1	<i>Deployment Cloud</i> (<i>Vercel</i> dan <i>Neon</i>)	83
4.10.2	Instalasi Manual (<i>On-Premise</i>)	83
4.10.3	Instalasi Berbasis Kontainer (<i>Docker</i>)	85
4.10.4	Kebutuhan Perangkat untuk Instalasi Mandiri	86

4.10.5	Isolasi Kontainer dan Koeksistensi dengan Aplikasi Lain	86
4.11	Pemenuhan Permintaan Dosen Pemilik Proyek	88
4.11.1	Permintaan yang Belum Tercapai	91
5	PENUTUP	92
5.1	Kesimpulan	92
5.2	Saran	94
	DAFTAR PUSTAKA	97

DAFTAR TABEL

1.1	Defisiensi Sistem CHP Terdahulu	5
1.2	Tujuan Kerja Praktik dan Status Implementasi	12
1.3	Jadwal Pelaksanaan Kerja Praktik	18
2.1	Ringkasan Skema Basis Data Platform CHP	28
2.2	Pemetaan Metodologi Pengujian Platform CHP	33
2.3	Contoh Skenario <i>Unit Test</i> pada <i>Scoring Engine</i>	37
3.1	Kebutuhan Pengguna Platform CHP	46
3.2	Kebutuhan Perangkat Lunak	47
3.3	Kebutuhan Perangkat Keras	47
3.4	Kebutuhan Fungsional Platform CHP	49
3.5	Kebutuhan Non-Fungsional Platform CHP	50
3.6	Inventaris Tabel Basis Data Platform CHP	54
3.7	Definisi <i>Enum</i> PostgreSQL	58
3.8	Prosedur API Publik (Alur Asesmen)	64
3.9	Prosedur API Pengguna Terdaftar	64
3.10	Prosedur API Panel Administrasi	65
4.1	Inventaris <i>File</i> Pengujian Unit	79
4.2	Kebutuhan Perangkat Server untuk Instalasi Mandiri	86
4.3	Perbandingan Jalur <i>Deployment Cloud</i> dan Instalasi Mandiri	88

DAFTAR GAMBAR

1.1	Gambaran Umum Arsitektur Platform Asesmen Digital CHP . .	7
2.1	Diagram Arsitektur Tiga Lapisan Platform Asesmen Digital CHP	25
2.2	Alur Autentikasi dan Otorisasi Platform CHP (<i>Sequence Diagram</i>)	29
2.3	Piramida Pengujian Platform Asesmen Digital CHP	34
2.4	Diagram Alur <i>Pipeline</i> CI/CD Platform CHP (<i>Activity Diagram</i>)	39
3.1	Diagram <i>Use Case</i> Platform Asesmen Digital CHP	48
3.2	Diagram Komponen Platform Asesmen Digital CHP	52
3.3	Diagram <i>Entity-Relationship</i> Klaster Inti (Asesmen & Pengguna).	59
3.4	Diagram <i>Entity-Relationship</i> Klaster Administratif.	61
3.5	Diagram Alur Layar Platform CHP	66

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Center for Health Psychology (CHP) adalah unit riset di bawah Program Studi Psikologi, Universitas Kristen Krida Wacana (UKRIDA), yang berperan sebagai pusat rujukan dalam bidang kesehatan mental dan penelitian psikologi klinis. Untuk menjalankan misi tersebut, CHP membutuhkan platform digital yang kokoh, bukan sekadar situs kuis daring, melainkan sebuah instrumen penelitian digital yang harus memenuhi standar validitas dan reliabilitas psikometrik. Prinsip fundamental ini menjadi landasan seluruh keputusan perancangan dalam proyek Kerja Praktik (KP) ini.

Sebelum proyek ini dimulai, CHP menggunakan tiga metode manual untuk melaksanakan asesmen psikologi. **Pertama**, metode *paper-based*: responden mengisi lembar jawaban secara manual dan administrator menghitung skor menggunakan panduan penilaian (*scoring guide*) masing-masing instrumen. **Kedua**, metode *spreadsheet* Excel: administrator mewawancarai responden dan memasukkan data ke dalam lembar kerja Excel; beberapa instrumen telah memiliki rumus otomatis, namun pengelolaan data secara keseluruhan tetap bersifat manual. **Ketiga**, metode Google Forms: pengumpulan jawaban dilakukan secara daring, namun data harus diekspor terlebih dahulu dan diproses secara manual di luar sistem. Tidak satu pun dari ketiga metode tersebut mampu menyimpan, mengarsipkan, dan memproses data asesmen secara terintegrasi dan otomatis.

Kondisi ini menimbulkan empat masalah operasional utama:

1. Proses penilaian yang lambat dan rentan terhadap kesalahan manusia (*human error*), terutama pada instrumen dengan banyak butir soal dan subskala.
2. Data asesmen tidak tersimpan secara terpusat dan terstruktur, sehingga sulit diakses untuk keperluan penelitian longitudinal.
3. Tidak tersedia fitur ekspor data berkualitas riset yang kompatibel dengan perangkat lunak analisis statistik seperti SPSS atau JASP.
4. Asesmen jarak jauh (*remote assessment*) tidak dapat dilaksanakan secara efektif karena ketergantungan pada proses manual.

Platform yang dikembangkan dalam proyek KP ini dirancang untuk secara bertahap mengakomodasi seluruh alur kerja asesmen. Pada iterasi saat ini, responden dapat mengisi asesmen secara daring (*online*) maupun di lokasi (*on-site*) melalui perangkat yang terhubung. Seluruh komputasi skor dilakukan secara instan oleh *scoring engine* tanpa intervensi manual. Sebagai rancangan pengembangan lanjutan (*future work*), platform ini nantinya akan memungkinkan administrator untuk mengimpor data asesmen manual (baik dari lembar *paper-based* maupun format Excel) sehingga hasil metode tradisional tetap dapat diintegrasikan ke dalam basis data terpusat.

Platform ini terdiri atas beberapa modul terintegrasi: modul asesmen yang mengelola siklus hidup pengerjaan tes mulai dari persetujuan peserta hingga komputasi skor; modul administrasi yang menyediakan antarmuka bagi tim psikologi untuk

mengelola instrumen, memantau statistik, dan mengekspor data riset; modul autentikasi yang mengimplementasikan sistem keamanan berlapis; serta modul pelaporan yang menghasilkan dokumen hasil asesmen dan mengirimkannya melalui surel.

Platform ini mengimplementasikan empat instrumen psikometri yang menjadi inti proses asesmen, sebagaimana diuraikan berikut.

Perceived Stress Scale (PSS-10) merupakan instrumen yang dikembangkan oleh Cohen, Kamarck, dan Mermelstein (1983) untuk mengukur sejauh mana situasi dalam kehidupan dinilai sebagai penuh tekanan, tidak terduga, dan di luar kendali selama satu bulan terakhir. Instrumen ini terdiri dari sepuluh butir dengan skala Likert lima poin (0–4) dan mencakup dua dimensi, yaitu ketidakberdayaan (*perceived helplessness*) dan efikasi diri (*perceived self-efficacy*); struktur dua faktor ini telah dikonfirmasi pada versi Bahasa Indonesia (Hakim dkk., 2024). Skor total berkisar antara 0–40, dengan skor yang lebih tinggi menunjukkan tingkat stres yang lebih besar.

Generalized Problematic Internet Use Scale 2 (GPIUS-2) dikembangkan oleh Caplan (2010) berdasarkan model kognitif-perilaku untuk mengukur penggunaan internet bermasalah secara umum. Instrumen ini terdiri dari lima belas butir yang mencakup lima dimensi, yaitu preferensi interaksi sosial daring (POSI), regulasi suasana hati, preokupasi kognitif, penggunaan kompulsif, dan dampak negatif, serta satu dimensi orde-kedua, disregulasi diri, yang merupakan gabungan preokupasi kognitif dan penggunaan kompulsif. Platform mengimplementasikan adaptasi Bahasa Indonesia dengan skala Likert lima poin (1–5), mengikuti versi adaptasi Indonesia yang memodifikasi skala delapan poin asli menjadi lima poin (lih. Reynaldo & Sokang,

2016).

Simplified Resilience Scale (SRS) merupakan ukuran resiliensi psikologis yang dikembangkan oleh Manning, Carr, dan Kail (2016), yang disusun dari butir-butir *Satisfaction With Life Scale* (Diener dkk., 1985) dan *Pearlin Mastery Scale* (Pearlin & Schooler, 1978) dengan mengacu pada kerangka *Resilience Scale* (Wagnild & Young, 1993). Versi asli terdiri dari dua belas butir; platform mengimplementasikan adaptasi yang menggunakan sebelas butir dengan skala Likert enam poin (1–6), dikelompokkan secara deskriptif ke dalam tiga aspek (efikasi, kepuasan, dan kontrol), dengan lima butir (1, 3, 5, 7, 11) diberi skor terbalik. Skor yang lebih tinggi menunjukkan resiliensi yang lebih baik.

Self-Reporting Questionnaire (SRQ-29) dikembangkan oleh *World Health Organization* (Beusenberg & Orley, 1994) sebagai instrumen skrining gangguan mental emosional di layanan kesehatan primer. Kuesioner ini terdiri dari dua puluh sembilan butir dengan format respons biner (Ya/Tidak) yang mencakup empat domain, yaitu gejala mental emosional atau GME (butir 1–20), penggunaan zat psikoaktif (butir 21), gejala psikotik (butir 22–24), dan gejala stres pascatrauma (butir 25–29). Validasi nasional instrumen ini dalam konteks Indonesia sebagian besar difokuskan pada versi 20 butir (SRQ-20) untuk mendeteksi gejala depresi (Idaiani dkk., 2019), namun platform ini secara operasional mengimplementasikan versi penuh 29 butir sesuai kebutuhan spesifik asesmen menyeluruh di CHP.

Contoh lengkap butir untuk keempat instrumen tersebut disajikan secara berturut-turut pada Lampiran C hingga Lampiran F.

Evaluasi teknis mendalam terhadap sistem web CHP yang sebelumnya dikem-

bangkan oleh tim mahasiswa mengungkapkan sejumlah defisiensi fundamental. Sistem lama dibangun menggunakan *HyperText Markup Language* (HTML), *Cascading Style Sheets* (CSS), dan JavaScript (JS) murni tanpa *framework* apapun, sehingga tidak memiliki struktur arsitektur yang memadai untuk kebutuhan riset jangka panjang. *Technical debt* (akumulasi dampak negatif dari keputusan desain yang tidak optimal) pada sistem lama bersifat struktural, bukan superfisial. Akibatnya, upaya perbaikan tambal sulam akan memakan lebih banyak sumber daya dibandingkan pembangunan ulang dari nol, namun tetap menghasilkan produk yang inferior. Tabel 1.1 merangkum seluruh defisiensi yang ditemukan beserta dampak operasionalnya.

Tabel 1.1: Defisiensi Sistem CHP Terdahulu

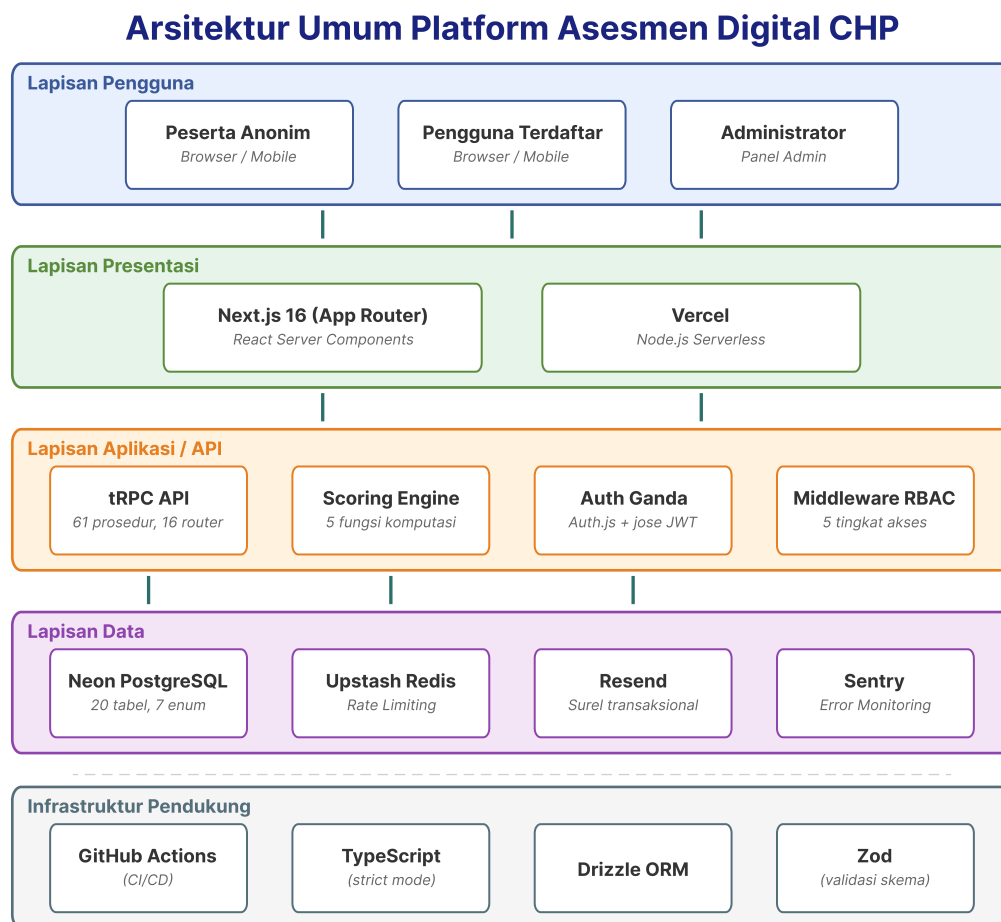
Masalah	Deskripsi	Dampak
Tanpa <i>Framework</i>	HTML/CSS/JS murni tanpa struktur arsitektur	Tidak dapat di- <i>scale</i> , dipelihara, atau diuji secara sistematis
Struktur File Datar	Semua <i>file</i> di direktori root tanpa pemisahan <i>concerns</i>	Setiap modifikasi berisiko menyebabkan kegagalan berantai di bagian lain
Tanpa <i>Type Safety</i>	JavaScript biasa tanpa <i>type checking</i> atau <i>linting</i>	<i>Bug</i> tersembunyi mencapai produksi tanpa terdeteksi
Cakupan Tes Nol	Tidak ada <i>unit test</i> , <i>integration test</i> , maupun <i>end-to-end test</i>	<i>Bug</i> regresi tidak terelakkan pada setiap perubahan kode
Desain Basis Data Tidak Memadai	Skema tidak dapat memodelkan penilaian multidimensional bervariasi	Memblokir fungsionalitas riset inti
Tanpa Dokumentasi	Tidak ada README, dokumentasi API, atau panduan <i>setup</i>	Setiap kontributor baru memulai dari nol tanpa acuan

Berdasarkan temuan pada Tabel 1.1, solusi yang dipilih adalah pembangunan

ulang total (*full redevelopment*) menggunakan *tech stack* modern berstandar industri. Platform baru dibangun di atas Next.js 16 sebagai *meta-framework* berbasis React, TypeScript sebagai bahasa pemrograman utama yang memberikan keamanan tipe (*type safety*), PostgreSQL sebagai Sistem Manajemen Basis Data Relasional (*Relational Database Management System/RDBMS*), tRPC untuk lapisan API yang aman secara tipe dari ujung ke ujung (*end-to-end type safety*), serta Drizzle *Object-Relational Mapping* (ORM) sebagai lapisan abstraksi basis data. Seluruh sistem di-*deploy* pada platform *serverless* Vercel dengan basis data terkelola melalui Neon PostgreSQL.

Sistem yang dibangun melayani empat aktor pengguna utama: (1) peserta anonim yang mengambil asesmen secara instan; (2) pengguna terdaftar yang dapat melacak riwayat asesmen; (3) administrator (termasuk tim riset psikologi) yang mengelola instrumen dan mengeksport data; serta (4) super administrator yang mengelola keamanan dan seluruh akun dalam sistem. Metodologi pengembangan yang digunakan adalah Agile iteratif dengan siklus *sprint* dua mingguan, memungkinkan umpan balik berkelanjutan dari pemangku kepentingan di setiap tahap pengembangan. Gambar 1.1 menampilkan gambaran umum arsitektur sistem platform CHP yang dirancang dalam proyek ini.

Gambar 1.1 memperlihatkan bagaimana keempat aktor mengakses satu basis kode yang sama melalui lapisan-lapisan yang terpisah; rincian teoretis setiap lapisan dibahas pada Bab 2, sedangkan penjabaran rancangannya diuraikan pada Bab 3.



Gambar 1.1: Gambaran Umum Arsitektur Platform Asesmen Digital CHP

1.2 Skema Kerja Praktik

Kerja Praktik (KP) adalah mata kuliah wajib pada Program Studi Informatika UK-RIDA yang menekankan aspek kemandirian mahasiswa dalam beradaptasi dengan dunia kerja nyata. Pelaksanaan KP dimaksudkan agar mahasiswa memperoleh pengalaman praktis dalam mengaplikasikan keahliannya di bidang teknologi informasi sekaligus memahami bagaimana suatu instansi dikelola.

KP ini dilaksanakan menggunakan skema Proyek Dosen, yaitu skema di mana mahasiswa terlibat langsung dalam pengembangan produk inovatif milik dosen selaku pemilik proyek. Dosen sebagaimana dimaksud adalah Ibu Astin selaku Ketua Program Studi Psikologi yang sekaligus menjabat sebagai Kepala Riset CHP UKRIDA. Skema Proyek Dosen dipilih karena pengembangan Platform Asesmen Digital CHP merupakan inisiatif riset resmi CHP yang membutuhkan produk *Information Technology* (IT) berupa platform asesmen psikometri digital berkualitas produksi (*production-ready*).

Dalam kerangka proyek ini, penulis bertanggung jawab atas tiga ranah pengembangan utama, yaitu:

1. Arsitektur sistem: perancangan skema basis data, infrastruktur *Continuous Integration/Continuous Deployment* (CI/CD), dan konfigurasi layanan *cloud*.
2. *Backend* dan API: implementasi prosedur *tRPC*, *scoring engine*, dan sistem autentikasi.
3. Pengujian dan implementasi (*testing and implementation*).

Pengembangan komponen antarmuka pengguna (*User Interface/UI*) dan desain pengalaman pengguna (*User Experience/UX*) dikerjakan secara terpisah oleh anggota tim lain dalam laporan KP tersendiri, sehingga kedua laporan saling melengkapi untuk menggambarkan sistem secara menyeluruh.

1.3 Metodologi Pengembangan Berbantuan Kecerdasan Buatan

Perlu dinyatakan secara transparan sejak awal bahwa seluruh proses pengembangan dalam proyek KP ini dilaksanakan dengan pendekatan *agentic engineering*: penulis mengarahkan agen kecerdasan buatan (AI) berbasis model bahasa besar untuk menghasilkan sebagian besar kode implementasi, sementara arah pengembangan, spesifikasi, keputusan teknis, dan verifikasi hasil sepenuhnya berada di tangan penulis. Pendekatan ini dipilih secara sadar sebagai strategi untuk mencapai target platform berkualitas produksi dalam batas waktu 12–14 minggu; landasan teori dan bukti empiris produktivitasnya diuraikan pada Subbab 2.1.2.

Dalam pola kerja ini, AI berperan sebagai alat produksi kode, bukan pengganti proses rekayasa. Kompetensi dan kontribusi teknis penulis terkonsentrasi pada empat ranah yang justru tidak dapat didelegasikan kepada AI:

1. **Keputusan arsitektur dan desain.** Pemilihan *tech stack* (Next.js, TypeScript, tRPC, Drizzle ORM, PostgreSQL), perancangan skema basis data 20 tabel (termasuk pemisahan `answers` dari `results` demi reproduktibilitas ilmiah, isolasi PII, dan strategi perlindungan data bertingkat), serta arsitektur autentikasi ganda yang memisahkan total sistem admin dari pengguna publik, seluruhnya merupakan keputusan penulis yang

ditetapkan *sebelum* kode dihasilkan (lih. Bab 3).

2. **Dekomposisi tugas** (*task decomposition*). Proyek dipecah menjadi fase-fase berjenjang (Tabel 1.3) dan setiap fase dipecah lagi menjadi unit tugas yang berbatas jelas, dapat diverifikasi, dan berukuran cukup kecil untuk ditinjau menyeluruh. Kualitas dekomposisi ini menentukan kualitas keluaran agen; tugas yang ambigu menghasilkan kode yang salah arah.
3. **Tinjauan dan validasi keluaran AI**. Tidak ada satu pun perubahan yang diintegrasikan tanpa melewati gerbang verifikasi otomatis (ESLint, pemeriksaan tipe *strict*, *build* produksi, dan 213 kasus uji unit pada *pipeline* CI) serta tinjauan manual penulis. Untuk aspek klinis, penulis menyusun dokumen audit per instrumen yang memeriksa kesesuaian implementasi terhadap spesifikasi psikometri dan literatur sumber.
4. **Deteksi dan koreksi kesalahan AI**. Proses audit pada ranah ketiga berulang kali menemukan keluaran AI yang keliru dan harus dikoreksi oleh penulis; contoh-contoh konkretnya diuraikan di bawah.

Tiga temuan audit berikut menjadi bukti bahwa keluaran AI tidak pernah diterima tanpa verifikasi. **Pertama**, ambang batas interpretasi GPIUS-2 yang semula dihasilkan AI (15–34/35–52/53–75) ternyata tidak memiliki sumber publikasi yang dapat ditelusuri pada studi validasi mana pun; penulis menggugurkan ambang tersebut dan menggantinya dengan skema berjangkar rata-rata (*mean-anchored*) yang merujuk sampel Jakarta pada Reynaldo dan Sokang (2016). **Kedua**, audit visualisasi menemukan komponen hasil menampilkan skor subskala turunan DSR dengan penyebut

yang salah (18/18, seolah 100%, alih-alih 18/30) karena nilai maksimum teoretis per dimensi tidak dipersistensi; koreksinya menuntut penelusuran rantai data lintas enam berkas dari *scoring engine* hingga komponen grafik. **Ketiga**, audit keamanan menyeluruh menemukan celah injeksi formula *spreadsheet* pada fitur ekspor CSV/XLSX serta prosedur mutasi permintaan laporan yang belum dibatasi peran; keduanya diperbaiki dan dilindungi uji regresi. Ketiganya adalah kelas kesalahan yang hanya terdeteksi oleh pemahaman domain (psikometri, aliran data, dan keamanan) yang menjadi tanggung jawab penulis, bukan agen.

Dengan demikian, penggunaan AI dalam proyek ini memindahkan titik berat kompetensi dari penulisan kode baris demi baris menuju perancangan sistem, dekomposisi masalah, dan penjaminan mutu. Meski demikian, tanggung jawab akhir atas kebenaran, keamanan, dan kualitas sistem tetap sepenuhnya berada pada penulis.

1.4 Tujuan dan Manfaat

Pelaksanaan KP ini memiliki enam tujuan yang dirumuskan menggunakan pendekatan SMART (*Specific, Measurable, Achievable, Relevant, Time-bound*), yaitu spesifik, terukur, dapat dicapai, relevan, dan berbatas waktu. Tabel 1.2 menyajikan rumusan tujuan beserta target terukur dan status implementasi pada saat laporan ini disusun.

Tabel 1.2: Tujuan Kerja Praktik dan Status Implementasi

No.	Tujuan	Target Terukur	Status
1	Membangun platform asesmen modern dan responsif sesuai standar aksesibilitas	Skor <i>Lighthouse Performance</i> ≥ 90 ; <i>Accessibility</i> ≥ 95 pada semua halaman utama	Selesai: Fase 1A & 1B (Minggu 1–6)
2	Meng-implementasikan <i>scoring engine</i> modular otomatis untuk perhitungan sumatif, berbobot, terbalik, dimensional, klaster biner, dan agregasi hirarkis	Skor komputasi sesuai nilai referensi manual; <i>engine</i> diuji dengan 59 kasus uji	Selesai: Fase 1B (Minggu 3–6)
3	Menyajikan umpan balik visual instan berupa skor total, <i>breakdown</i> dimensi, dan grafik	Halaman hasil <i>ter-render</i> dalam dua detik setelah pengiriman tes	Selesai: Fase 1B (Minggu 5–6)

No.	Tujuan	Target Terukur	Status
4	Membangun fungsionalitas ekspor data berkualitas riset dalam format CSV	Prosedur adminResults.export dengan batas 5000 baris, mencakup jawaban per butir soal	Selesai: Fase 1C (Minggu 7–10)
5	Mengembangkan panel administrator dengan RBAC	Tim psikologi dapat membuat, mengonfigurasi, dan mempublikasikan instrumen baru tanpa intervensi pengembang; empat peran admin	Selesai: Fase 1C (Minggu 7–10)
6	Memastikan serah terima sistem yang berkelanjutan dengan dokumentasi komprehensif	Pengembang baru dapat <i>clone</i> , <i>setup</i> , dan menjalankan proyek secara lokal dalam waktu 15 menit	Selesai: Fase 1D (Minggu 11–13)

Tercapainya sebagian besar tujuan pada Tabel 1.2 tersebut membawa manfaat yang diharapkan dari pengembangan Platform Asesmen Digital CHP (dengan catatan bahwa sebagian target teknis masih berupa desain arsitektur yang menunggu

pengujian operasional lanjutan), mencakup empat dimensi utama:

1. Validitas riset yang meningkat berkat penyimpanan respons mentah yang dijaga integritasnya melalui penguncian status sesi asesmen, disertai metadata versi instrumen dan versi algoritma penilaian untuk reproduktibilitas ilmiah.
2. Pengalaman pengguna yang profesional dan aksesibel dengan antarmuka yang menenangkan (*calm design*) sehingga meminimalkan kecemasan peserta tes.
3. Skalabilitas sistem: untuk menambahkan instrumen psikologi baru secara mandiri melalui panel admin tanpa memerlukan intervensi pengembang.
4. Keberlanjutan operasional lintas semester: berkat arsitektur yang terdokumentasi secara komprehensif dan kode yang aman secara tipe.

1.5 Batasan Kerja Praktik

Agar pengerjaan proyek tetap terfokus dan dapat diselesaikan dalam batas waktu yang ditentukan, ruang lingkup pekerjaan ditetapkan sebagai berikut.

Pekerjaan yang termasuk dalam ruang lingkup KP penulis adalah:

1. Perancangan dan implementasi skema basis data relasional menggunakan Drizzle ORM yang mendukung instrumen psikometri multidimensional, manajemen sesi anonim, dan jejak audit administratif.

2. Pengembangan infrastruktur CI/CD otomatis menggunakan GitHub Actions dengan dua *job* paralel: *job* pertama menjalankan *linting* (ES-Lint), *type checking* (TypeScript Compiler), dan *build* produksi; *job* kedua menjalankan *unit testing* (Vitest).
3. Implementasi *scoring engine* modular yang mendukung algoritma penilaian sumatif, berbobot, terbalik (*reversed*), dimensional, klaster biner, dan agregasi hirarkis (DSR), sepenuhnya dikonfigurasi melalui basis data.
4. Pengembangan prosedur API berbasis tRPC untuk manajemen sesi asesmen, penyimpanan jawaban, komputasi skor, autentikasi administrator, dan manajemen instrumen.
5. Implementasi sistem autentikasi ganda: Auth.js v5 untuk pengguna publik dan JWT mandiri melalui jose untuk panel administrator, dengan *Role-Based Access Control* (RBAC) lima tingkat.
6. Pengujian pada level unit dan prosedur, mencakup *unit test* pada *scoring engine* dan prosedur API, *integration test* prosedur API, serta pengujian keamanan dasar menggunakan *checklist OWASP Top Ten*. Pengujian *end-to-end* otomatis dan uji beban merupakan pekerjaan lanjutan.

Pekerjaan yang tidak termasuk dalam ruang lingkup KP ini adalah:

1. Pengembangan aplikasi *mobile native* (iOS/Android): platform bersifat berbasis web (*web-based*) dan responsif untuk perangkat seluler.

2. Interpretasi hasil berbasis kecerdasan buatan (AI) atau *Large Language Model* (LLM).
3. Fungsionalitas diagnosis klinis: platform hanya menyediakan *self-assessment* mandiri, bukan diagnosis klinis.
4. Pengembangan komponen UI dan desain UX, yang merupakan tanggung jawab anggota tim lain dalam laporan KP terpisah.
5. Fitur aksesibilitas khusus seperti dukungan *screen reader* atau teknologi asistif untuk penyandang disabilitas: fitur ini berada di luar cakupan versi ini dan dicatat sebagai batasan sistem yang dapat dikembangkan pada iterasi berikutnya.
6. Verifikasi usia responden: sistem tidak membatasi usia pengguna dan tidak dapat memverifikasi apakah responden di bawah umur didampingi oleh orang dewasa. Ini merupakan batasan yang disadari (*known limitation*) dan pengguna diasumsikan mampu membaca instruksi secara mandiri serta mengoperasikan perangkat masukan.
7. Fitur impor data asesmen historis: migrasi data dari sistem lama ke sistem baru direncanakan sebagai pekerjaan lanjutan (*future work*) dan tidak termasuk dalam cakupan kerja praktik ini.

Instrumen *Simplified Resilience Scale* (SRS) yang digunakan merupakan adaptasi (sebelas dari dua belas butir asli, dengan skala enam poin) dari instrumen Manning dkk. (2016) dan belum melalui proses validasi psikometrik formal dalam konteks Bahasa Indonesia. Penggunaannya pada platform bersifat indikatif sebagai gambaran

awal tingkat resiliensi, bukan sebagai alat diagnostik. Konten dan skala seluruh instrumen ditetapkan oleh dosen pemilik proyek selaku ahli domain; peran pengembang adalah mengimplementasikan spesifikasi tersebut secara akurat.

Selain batasan ruang lingkup di atas, dua batasan sistem pada aspek keamanan dan privasi data perlu dinyatakan secara eksplisit:

1. Data demografis disimpan terbaca (tidak dienkripsi per kolom).

Kolom nama, jenis kelamin, usia, dan domisili pada tabel `session_demographics` dikonsumsi secara fungsional oleh aplikasi: ditampilkan pada laporan hasil individual, serta dicari, difilter, diurutkan, dan diekspor melalui kueri SQL di panel admin. *Hashing* satu arah akan menghilangkan nilainya, dan enkripsi per kolom akan mematahkan kemampuan kueri tersebut. Perlindungannya karena itu bertumpu pada kontrol akses berbasis peran, jejak audit, pembekuan data setelah sesi selesai, dan enkripsi *at rest* pada infrastruktur Neon, bukan pada kriptografi per kolom. Justifikasi desain selengkapnya diuraikan pada Subbab 3.4, dan pseudonimisasi data demografis dicatat sebagai pekerjaan lanjutan pada Subbab 5.2.

2. Jaminan keamanan data bersifat relatif, bukan absolut.

Platform menerapkan pertahanan berlapis pada tingkat aplikasi (RBAC, *hashing* IP, enkripsi surel AES-256-GCM, *rate limiting*, jejak audit) dan mengandalkan jaminan keamanan infrastruktur dari penyedia layanan tersertifikasi (Neon: enkripsi *at rest* dan *in transit*, SOC 2 Type 2, ISO 27001). Namun, di tengah lanskap ancaman yang terus berevolusi (termasuk

serangan otomatis berbasis AI), tidak ada sistem yang dapat menjamin kerahasiaan data secara mutlak. Pengujian penetrasi formal dan audit keamanan oleh pihak ketiga berada di luar cakupan KP ini; risiko residual tersebut diakui sebagai batasan sistem (landasan teorinya pada Subbab 2.7.4).

1.6 Waktu Pelaksanaan

Pelaksanaan KP ini berlangsung selama kurang lebih 12 hingga 14 minggu, dimulai pada 11 Februari 2026 melalui koordinasi perdana dengan dosen pemilik proyek. Pengembangan dan serah terima sistem diselesaikan pada akhir Mei 2026, dan rangkaian KP ditutup dengan sidang yang dilaksanakan pada 22 Juli 2026. Tabel 1.3 menyajikan rincian jadwal pelaksanaan per fase beserta kegiatan utama masing-masing.

Tabel 1.3: Jadwal Pelaksanaan Kerja Praktik

Fase	Periode	Kegiatan Utama
Fase 0: Perencanaan	11 Feb 2026	Koordinasi perdana, penyusunan rencana proyek (731 baris <i>unified project plan</i>), analisis sistem lama
Fase 1A: Fondasi & Setup	Minggu 1–2	Inisialisasi proyek Next.js, desain skema basis data (20 tabel), konfigurasi CI/CD, <i>seeding</i> data awal

Fase	Periode	Kegiatan Utama
Fase 1B: Inti Asesmen	Minggu 3–6	Katalog tes, mesin asesmen multi-langkah, <i>scoring engine</i> , prosedur tRPC, <i>pre-test</i> dan <i>post-test flow</i>
Fase 2A–2C: Autentikasi	Minggu 4–7	Infrastruktur JWT ganda, UI autentikasi (<i>sign-up</i> , <i>login</i>), mutasi <i>claim session</i> , migrasi skema Auth.js
Fase 1C: Admin & Polish	Minggu 7–10	Panel admin dengan RBAC empat peran, CRUD instrumen tes, konfigurasi penilaian, ekspor data
Fase 1D: Pengujian & Serah Terima	Minggu 11–13	Pengujian unit (213 kasus uji), pengujian keamanan (OWASP <i>checklist</i>), dokumentasi teknis, <i>deployment</i> final
Sidang KP	22 Juli 2026	Presentasi dan evaluasi akhir di hadapan tim penguji Program Studi Informatika UKRIDA

Fase-fase pada Tabel 1.3 sengaja disusun tumpang-tindih (misalnya fase autentikasi berjalan paralel dengan fase inti asesmen) mengikuti pola *sprint* Agile dua mingguan yang diuraikan pada Bab 2. Bab berikutnya memaparkan landasan teori yang mendasari seluruh keputusan teknis dalam fase-fase tersebut.

BAB 2

LANDASAN TEORI

Bab ini memaparkan landasan teori yang menjadi dasar ilmiah dan teknis dalam perancangan serta implementasi Platform Asesmen Digital CHP. Sesuai dengan fokus penulis pada arsitektur sistem, pengembangan *backend*, dan pengujian perangkat lunak, teori yang dibahas mencakup rekayasa perangkat lunak dengan metodologi Agile, arsitektur sistem web modern berbasis Next.js, pengembangan *backend* dengan tRPC dan PostgreSQL, sistem autentikasi dan otorisasi, strategi pengujian perangkat lunak, serta keamanan dan privasi data.

2.1 Rekayasa Perangkat Lunak

Rekayasa Perangkat Lunak (*Software Engineering*) adalah disiplin ilmu teknik yang berkaitan dengan semua aspek produksi perangkat lunak, mulai dari tahap spesifikasi awal sistem hingga pemeliharaan sistem setelah digunakan (Sommerville 2016). Rekayasa perangkat lunak mencakup manajemen proyek, proses pengembangan, metode teknis, dan penggunaan alat (*tools*) yang tepat guna menghasilkan perangkat lunak yang dapat diandalkan, efisien, dan memenuhi kebutuhan penggunanya.

Proses rekayasa perangkat lunak pada umumnya melibatkan empat aktivitas fundamental yang saling berkaitan, yaitu: (1) spesifikasi perangkat lunak, yaitu pen-
definisian fungsionalitas dan kendala operasi sistem; (2) pengembangan perangkat lunak, yaitu perancangan dan pemrograman sistem sesuai spesifikasi; (3) validasi perangkat lunak, yaitu pemastian bahwa sistem memenuhi kebutuhan pengguna;

serta (4) evolusi perangkat lunak, yaitu modifikasi sistem sebagai respons terhadap perubahan kebutuhan. Pada proyek Platform CHP, keempat aktivitas ini dilaksanakan secara iteratif dalam kerangka metodologi Agile.

2.1.1 Model Proses: Metodologi Agile

Model proses perangkat lunak mendefinisikan pendekatan yang digunakan untuk mengorganisasi dan mengelola aktivitas pengembangan. Dalam proyek ini, model Agile iteratif dan inkremental dipilih sebagai kerangka kerja pengembangan. Agile adalah sekumpulan nilai dan prinsip yang diformulasikan dalam *Agile Manifesto* (2001), yang menekankan respons terhadap perubahan di atas mengikuti rencana yang kaku, kolaborasi erat dengan pelanggan di atas negosiasi kontrak, dan penyerahan perangkat lunak yang berfungsi di atas dokumentasi yang ekstensif.

Manifestasi Agile yang digunakan dalam proyek ini adalah Scrum, sebuah kerangka kerja manajemen proyek yang mengorganisasi pekerjaan ke dalam siklus waktu tetap yang disebut *sprint* (Schwaber dan Sutherland 2020). Setiap *sprint* pada proyek CHP berdurasi dua minggu. Pada setiap akhir *sprint*, tim menghasilkan *increment* (versi perangkat lunak yang siap didemonstrasikan dan berpotensi untuk dirilis). Dalam konteks KP ini, pertemuan dua mingguan dengan Ibu Astin selaku pemilik proyek berfungsi sebagai sesi *sprint review*, di mana demonstrasi fitur yang telah diimplementasikan disampaikan untuk mendapatkan umpan balik langsung.

Pemilihan Agile untuk proyek ini didasari tiga pertimbangan utama:

1. Jadwal tetap 12 minggu menuntut penyerahan fitur secara inkremental, sehingga pemangku kepentingan dapat menggunakan dan memberikan

umpan balik terhadap fitur yang sudah selesai sembari fitur lain masih dikembangkan.

2. Kebutuhan instrumen psikometri dari tim psikologi dapat berevolusi selama pengembangan, dan Agile mengakomodasi perubahan ini tanpa memerlukan renegosiasi kontrak formal.
3. Tim kecil yang terdiri atas dua pengembang cocok dengan prinsip Agile yang mengutamakan komunikasi langsung dan minimalisasi birokrasi proses.

2.1.2 Pengembangan Perangkat Lunak Berbantuan Kecerdasan Buatan

Perkembangan model bahasa besar (*Large Language Model/LLM*) dalam beberapa tahun terakhir melahirkan praktik baru dalam rekayasa perangkat lunak, yaitu pengembangan berbantuan kecerdasan buatan (AI). Spektrum praktik ini membentang dari asisten pelengkap kode (*code completion*) hingga pendekatan yang lebih menyeluruh yang dikenal sebagai *agentic engineering*: pengembang mendelegasikan tugas implementasi yang terdefinisi jelas kepada agen AI, kemudian meninjau, menguji, dan mengoreksi keluarannya secara sistematis. Bukti empiris menunjukkan pendekatan ini berdampak nyata terhadap produktivitas; eksperimen terkontrol oleh Peng dkk. (2023) mencatat bahwa pengembang yang dibantu AI menyelesaikan tugas implementasi 55,8% lebih cepat dibandingkan kelompok kontrol.

Namun, literatur yang sama menegaskan bahwa percepatan tersebut tidak menghilangkan peran pengembang, melainkan menggesernya. Keluaran model bahasa bersifat probabilistik: kode yang dihasilkan dapat mengandung kesalahan logika

yang halus, asumsi yang tidak berdasar, atau referensi yang tidak dapat ditelusuri sumbernya. Oleh karena itu, penggunaan AI dalam pengembangan perangkat lunak yang bertanggung jawab menuntut tiga prasyarat: (1) spesifikasi dan dekomposisi tugas yang jelas dari pengembang; (2) jalur verifikasi otomatis (pengujian unit, pemeriksaan tipe, *linting*) yang menyaring keluaran sebelum diintegrasikan; dan (3) tinjauan manual oleh pengembang yang memahami domain masalah. Dalam kerangka empat aktivitas fundamental Sommerville (2016), AI mengambil porsi terbesar pada aktivitas pengembangan (pemrograman), sementara spesifikasi dan validasi justru semakin bertumpu pada kompetensi pengembang. Penerapan konkret prinsip ini pada proyek CHP, beserta pembagian peran antara penulis dan agen AI, diuraikan pada Subbab 1.3.

2.2 Arsitektur Sistem Web Modern

Setelah model proses pengembangan ditetapkan, keputusan fundamental berikutnya adalah arsitektur sistem. Arsitektur sistem web mengacu pada struktur tingkat tinggi dari sebuah aplikasi web yang mendefinisikan bagaimana komponen-komponen penyusunnya diorganisasi, berinteraksi, dan berkomunikasi satu sama lain. Bagi platform asesmen yang harus menjaga integritas data riset, pilihan arsitektur menentukan seberapa mudah sistem diuji, diamankan, dan dipelihara dalam jangka panjang.

2.2.1 Arsitektur Tiga Lapisan

Arsitektur tiga lapisan (*three-tier architecture*) adalah pola arsitektur yang membagi aplikasi menjadi tiga lapisan logis yang terpisah: lapisan presentasi (*presentation*

tier), lapisan aplikasi (*application tier*), dan lapisan data (*data tier*). Pemisahan ini menerapkan prinsip *separation of concerns* yang memungkinkan setiap lapisan untuk dikembangkan, diuji, dan diskalakan secara independen.

Pada Platform Asesmen Digital CHP, ketiga lapisan diimplementasikan sebagai berikut:

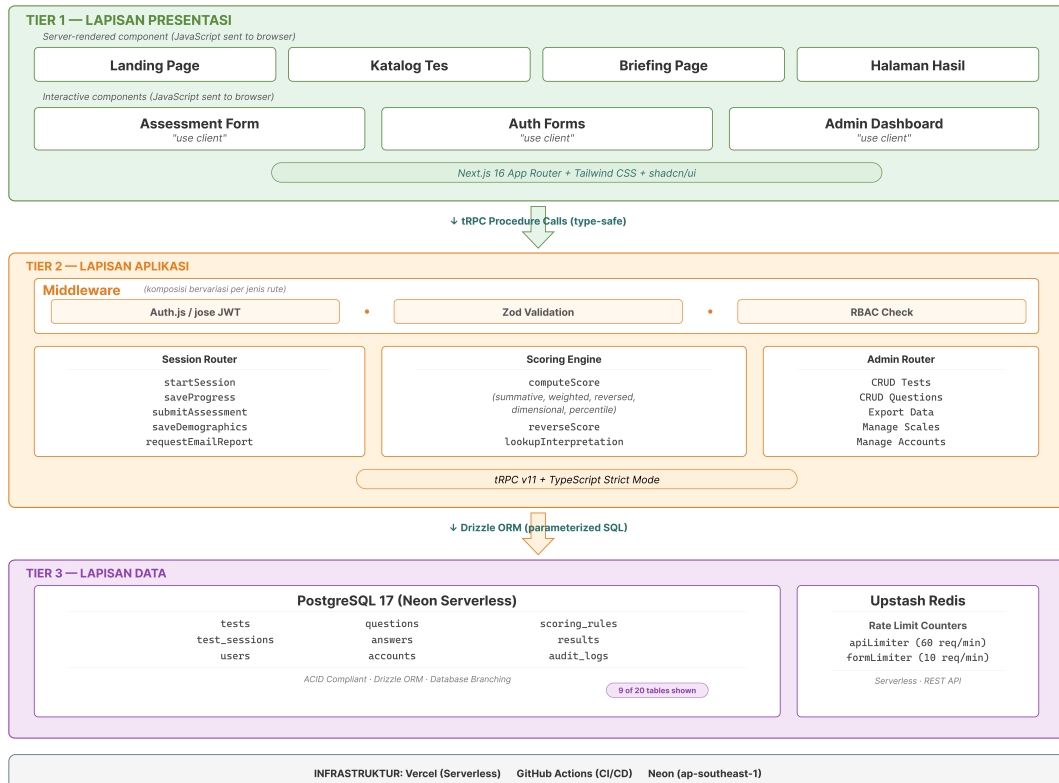
1. Lapisan Presentasi: Komponen React dengan *React Server Components* (RSC), Tailwind CSS, dan shadcn/ui yang menghasilkan antarmuka pengguna interaktif dan responsif.
2. Lapisan Aplikasi: *Router* tRPC, *scoring engine*, *middleware* autentikasi, dan *Server Actions* yang mengandung seluruh logika bisnis sistem.
3. Lapisan Data: PostgreSQL yang diakses melalui Drizzle ORM sebagai sumber data utama.

Gambar 2.1 mengilustrasikan diagram arsitektur tiga lapisan Platform CHP.

Lapisan mana yang benar-benar dieksekusi di server dan mana yang dikirim ke *browser* klien ditentukan oleh pilihan *meta-framework*; pada Platform CHP, keputusan ini jatuh pada Next.js.

2.2.2 Next.js dan React Server Components

Next.js adalah *meta-framework* berbasis React yang menyediakan fondasi lengkap untuk membangun aplikasi web *full-stack* modern (Vercel 2026). Proyek ini menggunakan Next.js 16 dengan *App Router* yang membawa pergeseran arsitektural signifikan melalui *React Server Components* (RSC).



Gambar 2.1: Diagram Arsitektur Tiga Lapisan Platform Asesmen Digital CHP

RSC adalah paradigma *rendering* yang memungkinkan komponen React dieksekusi dan di-*render* secara eksklusif di sisi server, tanpa mengirimkan kode JavaScript komponen tersebut ke *browser* klien. Implikasi praktis RSC untuk Platform CHP adalah pengurangan ukuran *bundle* JavaScript yang dikirim ke *browser* peserta tes. Komponen yang hanya membaca data, seperti halaman katalog tes, halaman informasi, dan halaman hasil, di-*render* di server tanpa mengirimkan kodenya ke klien. Hanya komponen yang memerlukan interaktivitas, seperti formulir asesmen dan tombol navigasi, yang ditandai dengan direktif "use client" dan dikirim ke *browser*.

2.3 TypeScript dan Keamanan Tipe

Arsitektur yang baik tidak banyak berarti bila kode yang mengisinya rapuh; di sinilah pemilihan bahasa pemrograman berperan. TypeScript adalah bahasa pemrograman *open-source* yang dikembangkan oleh Microsoft sebagai *superset* dari JavaScript. TypeScript menambahkan sistem pengetikan statis (*static type system*) di atas JavaScript, sehingga setiap variabel, parameter fungsi, dan nilai kembalian dapat diberikan anotasi tipe eksplisit.

2.3.1 Signifikansi TypeScript dalam Proyek CHP

Sebuah studi berskala besar terhadap 85 proyek web oleh Bogner dan Merkel (2022) menunjukkan bahwa penggunaan TypeScript menurunkan densitas *code smell* sebesar 34% dan mengurangi kompleksitas kognitif sebesar 28% dibandingkan JavaScript murni (Bogner dan Merkel 2022). Studi lain (arXiv 2026) mengungkap bahwa pengetikan statis mengubah lanskap kegagalan pada aplikasi modern: mayoritas *bug* dalam proyek ekosistem TypeScript bersumber dari kesalahan konfigurasi *toolchains*, penyalahgunaan integrasi antarmuka API, dan kegagalan penanganan status asinkron.

Pada Platform CHP, TypeScript digunakan di seluruh *codebase* dengan konfigurasi `strict: true` yang mengaktifkan pemeriksaan tipe paling ketat. Setiap prosedur tRPC mendefinisikan skema masukan dan keluaran menggunakan pustaka validasi Zod, yang secara otomatis menghasilkan tipe TypeScript.

2.4 Backend dan Basis Data

2.4.1 PostgreSQL dan Basis Data *Serverless*

PostgreSQL adalah sistem manajemen basis data relasional *open-source* yang dikenal karena kepatuhannya terhadap standar SQL, ekstensibilitas, dan dukungan terhadap tipe data kompleks seperti JSONB (PostgreSQL Global Development Group 2026). Platform CHP menggunakan PostgreSQL 17 melalui layanan *serverless* Neon yang memisahkan komputasi dari penyimpanan, memungkinkan basis data untuk diskalakan ke nol saat tidak ada permintaan dan aktif kembali dalam hitungan milidetik.

2.4.2 Drizzle ORM dan Perancangan Skema

Drizzle ORM adalah ORM TypeScript generasi baru yang berfilosofi “SQL-*first*”, sintaksis API-nya dirancang semirip mungkin dengan SQL namun sepenuhnya aman secara tipe (Drizzle Team 2026). Skema basis data Platform CHP dirancang untuk mendukung instrumen psikometri yang fleksibel. Tabel 2.1 menyajikan ringkasan tabel-tabel utama.

Keputusan arsitektur terpenting adalah pemisahan tabel *answers* dari tabel *results*. Dengan menyimpan respons mentah secara permanen dan terpisah dari skor yang telah dihitung, tim psikologi dapat melakukan *re-scoring* menggunakan algoritma yang diperbarui tanpa kehilangan data historis asli.

Tabel 2.1: Ringkasan Skema Basis Data Platform CHP

Nama Tabel	Kolom Kunci	Tujuan
tests	id, slug, title, version	Metadata instrumen psikometri dengan kontrol versi
questions	id, test_id, type, dimension, is_reversed, weight	Butir soal dengan informasi dimensi dan pembobotan
scoring_rules	id, test_id, algorithm, config	Skema placeholder (tidak digunakan)
test_sessions	id, test_id, status, ip_hash, claim_token	Sesi asesmen dengan mekanisme klaim
answers	id, session_id, question_id, value	Respons mentah yang <i>immutable</i>
results	id, session_id, total_score, dimension_scores	Skor terhitung yang terpisah dari respons mentah
audit_logs	id, admin_user_id, action, entity_type	Jejak audit untuk akuntabilitas

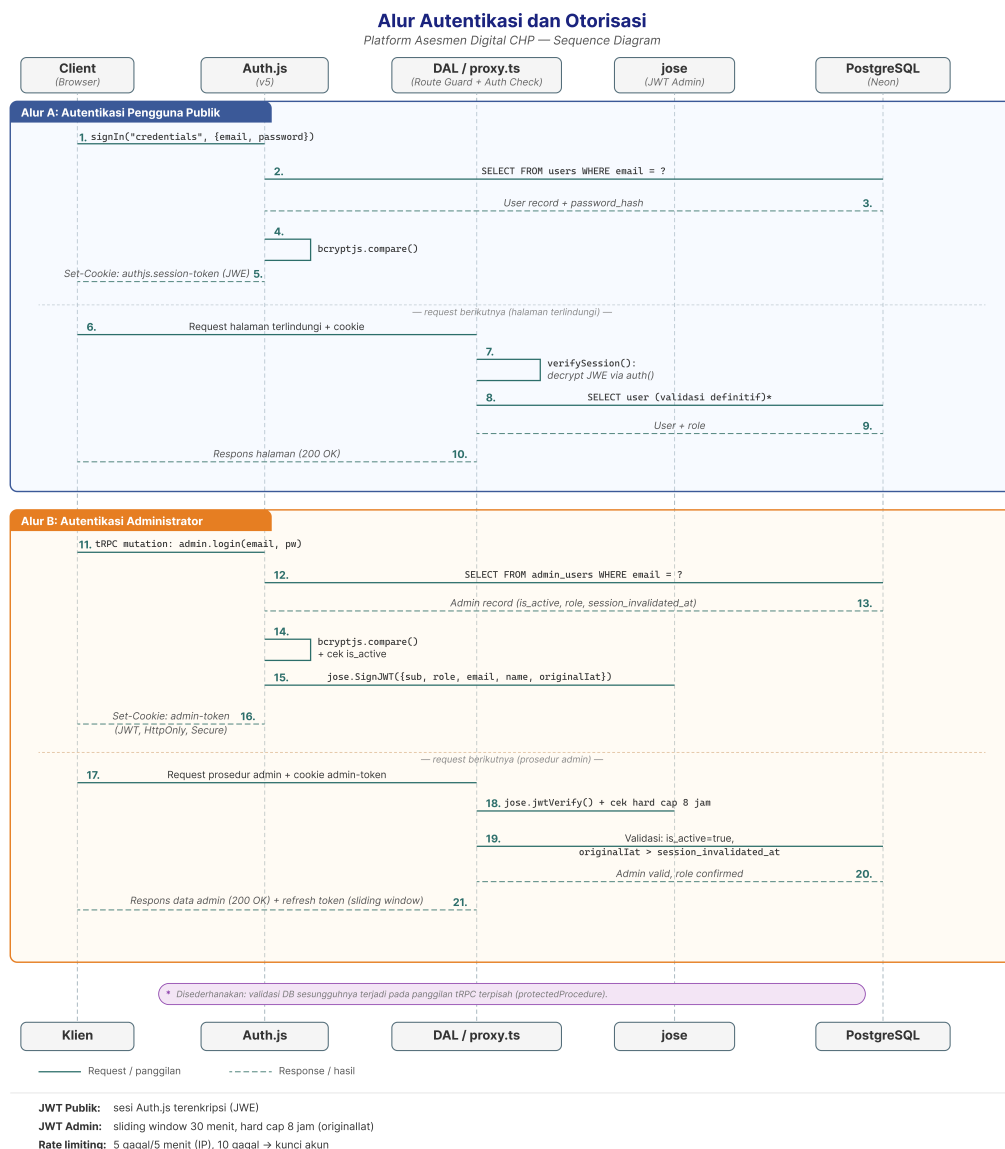
2.4.3 tRPC

tRPC (*TypeScript Remote Procedure Call*) adalah *framework* yang memungkinkan pembuatan API yang sepenuhnya aman secara tipe tanpa memerlukan generasi kode atau skema terpisah (tRPC Team 2026). Pada Platform CHP, prosedur tRPC diorganisasi ke dalam *router* berdasarkan domain bisnis:

1. Router sesi asesmen: Prosedur `startSession`, `saveProgress`, dan `submitAssessment` yang mengelola siklus hidup sesi tes.
2. Router autentikasi: Prosedur yang menangani registrasi, login, dan klaim sesi anonim ke akun terdaftar.
3. Router admin: Prosedur yang dilindungi oleh *middleware* RBAC untuk operasi CRUD instrumen tes, konfigurasi penilaian, dan ekspor data.

2.5 Autentikasi dan Otorisasi

Autentikasi (*authentication*) adalah proses verifikasi identitas pengguna, sedangkan otorisasi (*authorization*) menentukan apa yang diizinkan dilakukan oleh pengguna yang telah terautentikasi. Gambar 2.2 mengilustrasikan alur autentikasi dan otorisasi pada Platform CHP.



Gambar 2.2: Alur Autentikasi dan Otorisasi Platform CHP (*Sequence Diagram*)

2.5.1 JSON Web Token (JWT)

Komponen kunci yang membawa identitas pengguna di sepanjang alur pada Gambar 2.2 adalah *token*. *JSON Web Token* (JWT) adalah standar terbuka yang didefinisikan dalam RFC 7519 untuk mentransmisikan informasi secara aman antara pihak-pihak sebagai objek JSON yang ringkas dan mandiri (Jones, Bradley, dan Sakimura 2015). Sebuah JWT terdiri atas tiga bagian: Header (metadata *token*), Payload (klaim tentang pengguna), dan Signature (tanda tangan digital).

Platform CHP mengimplementasikan sistem autentikasi ganda berbasis JWT:

1. **Autentikasi pengguna publik** melalui Auth.js (NextAuth v5) dengan *CredentialsProvider*, di mana sesi pengguna disimpan dalam JWT terenkripsi dengan strategi sesi JWT.
2. **Autentikasi administrator** melalui pustaka jose secara mandiri (terpisah dari Auth.js), dengan *sliding window* 30 menit dan batas keras (*hard cap*) 8 jam dari waktu penerbitan awal. JWT administrator disimpan dalam *cookie* bernama `admin-token`.

2.5.2 Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) adalah model kontrol akses di mana izin diberikan kepada peran tertentu, dan pengguna mendapatkan izin melalui keanggotaan dalam peran yang relevan. RBAC menerapkan prinsip hak istimewa minimal (*principle of least privilege*).

Platform CHP mendefinisikan empat peran untuk pengguna panel admin melalui *enum* PostgreSQL `admin_role`:

1. **Super Admin** (`super_admin`): Memiliki akses penuh terhadap semua operasi, termasuk manajemen akun admin, konfigurasi sistem, dan kemampuan menghapus data.
2. **Admin** (`admin`): Memiliki akses untuk mengelola instrumen tes, melihat data respons sesi, dan mengeksport data penelitian, namun tidak dapat mengelola pengguna admin lain.
3. **Psikiater** (`psychiatrist`): Memiliki akses untuk melihat hasil asesmen dan menyetujui permintaan laporan, dengan fokus pada evaluasi klinis.
4. **Peneliti** (`researcher`): Memiliki akses baca terhadap data riset dan statistik untuk keperluan penelitian, tanpa kemampuan mengubah konfigurasi sistem.

Pemeriksaan otorisasi RBAC diimplementasikan pada lapisan *middleware* prosedur tRPC menggunakan lima tingkat kontrol akses:

1. `publicProcedure`: tidak memerlukan autentikasi. Digunakan untuk prosedur asesmen dan hasil.
2. `protectedProcedure`: memerlukan sesi `Auth.js` yang valid. Digunakan untuk operasi yang memerlukan identitas pengguna (klaim sesi, profil, riwayat).
3. `adminProcedure`: memvalidasi JWT admin melalui `jwt`, memeriksa status aktif dan waktu invalidasi sesi di basis data, serta menerapkan *sliding window refresh*. Digunakan untuk operasi baca pada panel admin.

4. `adminMutationProcedure`: memperluas `adminProcedure` dengan membatasi akses hanya kepada peran `super_admin` dan `admin`. Digunakan untuk operasi tulis pada instrumen tes dan konfigurasi.
5. `superAdminProcedure`: memperluas `adminProcedure` dengan membatasi akses secara eksklusif kepada peran `super_admin`. Digunakan untuk manajemen akun admin dan operasi keamanan.

2.6 Pengujian Perangkat Lunak

Pengujian perangkat lunak adalah proses evaluasi dan verifikasi bahwa produk perangkat lunak melakukan apa yang seharusnya dilakukan. Pengujian yang komprehensif amat penting untuk Platform CHP mengingat kekritisannya akurasi skor dalam konteks penelitian psikometri.

2.6.1 Metodologi Pengujian

Dua metodologi pengujian utama digunakan dalam proyek ini. *White-box testing* (pengujian kotak putih, juga dikenal sebagai pengujian struktural) adalah teknik pengujian di mana penguji memiliki akses penuh terhadap kode sumber dan struktur internal sistem (Capote 2023; Bhat dan Khan 2021). Penguji merancang kasus uji berdasarkan pengetahuan tentang jalur eksekusi, kondisi percabangan, dan aliran data dalam kode. *Black-box testing* (pengujian kotak hitam, juga dikenal sebagai pengujian fungsional atau pengujian perilaku) adalah teknik pengujian di mana penguji hanya mengetahui masukan dan keluaran yang diharapkan, tanpa akses terhadap implementasi internal (Capote 2023; Khan dan Khan 2022).

Platform CHP menggunakan pendekatan gabungan: *white-box testing* diterapkan pada *scoring engine* untuk memverifikasi setiap jalur komputasi (pembalikan skor, pengelompokan dimensional, penanganan kasus tepi), sedangkan *black-box testing* diterapkan pada alur pengguna secara keseluruhan (mengisi asesmen, menerima hasil, mengelola instrumen). Tabel 2.2 menyajikan pemetaan metodologi pengujian yang digunakan.

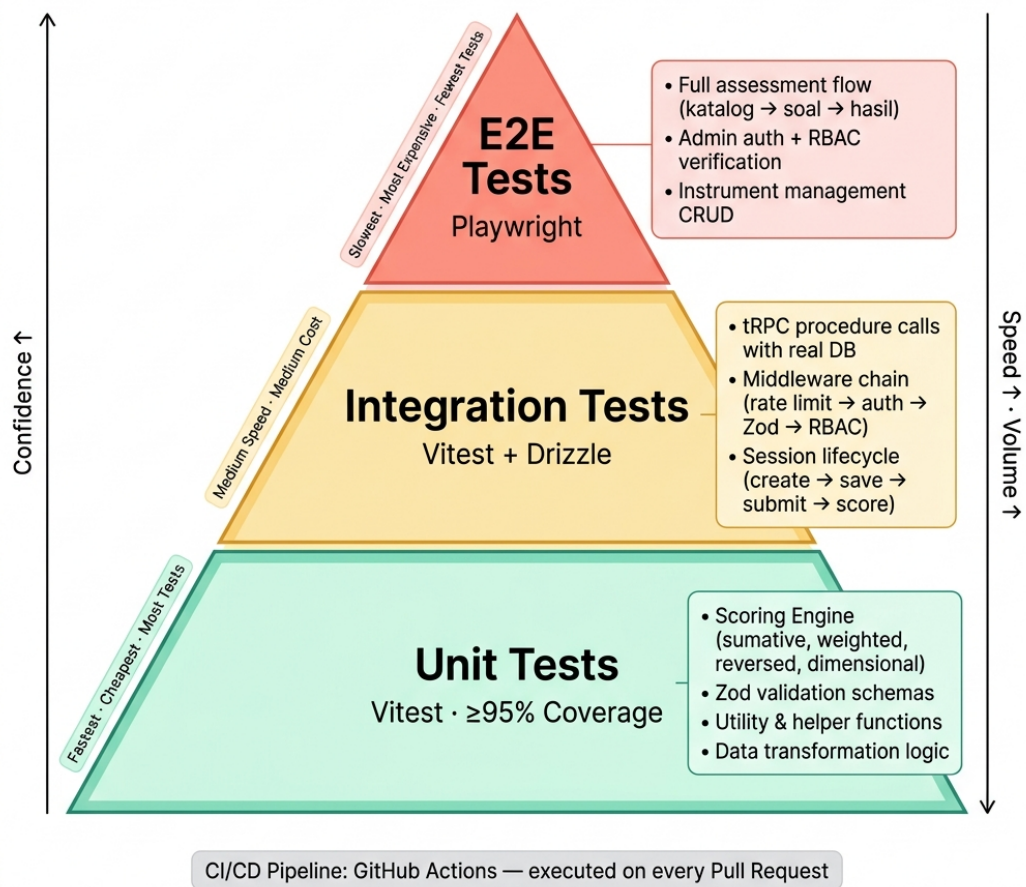
Tabel 2.2: Pemetaan Metodologi Pengujian Platform CHP

Metodologi	Level	Komponen	Alat
<i>White-box</i>	<i>Unit</i>	<i>Scoring engine</i> , validasi, enkripsi	Vitest
<i>White-box</i>	Integrasi	tRPC ↔ DB, Auth ↔ RBAC	Vitest
<i>Black-box</i>	<i>End-to-end</i>	Alur asesmen, alur admin	Playwright
<i>Black-box</i>	Validasi	Akurasi skor vs. panduan manual	Vitest

Pemetaan pada Tabel 2.2 memperlihatkan bahwa kedua metodologi tidak saling menggantikan, melainkan saling melengkapi: *white-box* menjamin kebenaran internal komputasi skor, sedangkan *black-box* menjamin bahwa pengalaman pengguna dari awal hingga akhir berperilaku sesuai harapan.

2.6.2 Strategi Piramida Pengujian

Strategi pengujian Platform CHP mengikuti konsep piramida pengujian (*testing pyramid*) yang dipopulerkan oleh Cohn (2009). Piramida pengujian menempatkan *unit test* pada dasar piramida (jumlah terbanyak, eksekusi tercepat, biaya terendah), *integration test* di tengah, dan *end-to-end test* di puncak (jumlah paling sedikit, eksekusi paling lambat, biaya tertinggi). Pendekatan ini memaksimalkan cakupan pengujian sembari meminimalkan waktu eksekusi dan biaya pemeliharaan (Desai, Singh, dan Amilkanthwar 2025). Gambar 2.3 mengilustrasikan hierarki ini.



Gambar 2.3: Piramida Pengujian Platform Asesmen Digital CHP

Lapisan *integration test* di tengah piramida pada Gambar 2.3 itulah yang diuraikan lebih rinci berikut ini.

2.6.3 Pengujian Prosedur (*Mock DB*)

Integration testing pada Platform CHP menggunakan metodologi *grey-box*: pengujian mengetahui arsitektur internal (skema basis data, kontrak tRPC) namun menguji prosedur sebagai unit fungsional yang utuh. Tiga area fokus integrasi:

1. **tRPC ↔ Basis Data**: memverifikasi bahwa prosedur tRPC berinteraksi secara benar dengan lapisan *mock* basis data, memastikan logika operasional sekuensial dieksekusi tepat tanpa memodifikasi data produksi, serta memverifikasi logika kondisional secara terisolasi.
2. **Autentikasi ↔ RBAC**: memverifikasi bahwa setiap tingkat *middleware* (dari *publicProcedure* hingga *superAdminProcedure*) secara benar mengizinkan atau menolak akses berdasarkan peran pengguna.
3. **Validasi Zod ↔ Logika Bisnis**: memverifikasi bahwa skema masukan Zod menolak data yang tidak valid sebelum logika bisnis dieksekusi, serta bahwa pesan kesalahan yang dikembalikan informatif dan konsisten.

2.6.4 Pengujian Validasi: Akurasi Skor Otomatis

Dalam konteks platform asesmen psikometri, pengujian validasi memiliki signifikansi yang melampaui pengujian perangkat lunak konvensional. Skor yang dihasilkan oleh *scoring engine* harus identik dengan skor yang dihitung secara manual menggunakan panduan penilaian (*scoring guide*) resmi masing-masing instrumen (Liu, Chen,

dan Sun 2016). Pengujian validasi dilaksanakan dengan membandingkan keluaran *scoring engine* terhadap tabel referensi yang dihitung secara manual untuk setiap instrumen, memverifikasi keidentikan skor (*score identity accuracy*). Selain akurasi, efisiensi waktu juga diukur: komputasi skor otomatis yang sebelumnya memerlukan hitungan menit secara manual kini diselesaikan dalam hitungan milidetik.

2.6.5 Unit Testing dengan Vitest

Vitest adalah *framework unit testing* berbasis Vite yang dirancang khusus untuk ekosistem TypeScript/JavaScript modern. Vitest menggunakan model eksekusi yang kompatibel dengan Jest namun dengan kecepatan yang lebih tinggi berkat eksekusi langsung tanpa proses kompilasi TypeScript berulang.

Modul *scoring engine* menjadi prioritas utama *unit testing* pada Platform CHP. Pipeline penilaian terdiri atas dua lapisan:

Inti Penilaian Murni (4 fungsi, 0 dependensi eksternal):

1. `reverseScore(rawValue, options)`: menghitung skor terbalik secara agnostik skala: $(scaleMax - rawValue) + scaleMin$.
2. `computeScore(input)`: fungsi inti yang menangani pembalikan butir soal, penjumlahan berbobot, pengelompokan dimensional, *rollup* orde kedua untuk DSR (GPIUS-2), dan komputasi `maxPossibleScore`.
3. `validateAnswerValues(answers, questions)`: memvalidasi bahwa setiap nilai jawaban berada dalam batas opsi yang valid.
4. `validateCompleteness(answers, questions)`: memvalidasi bahwa semua butir soal wajib telah dijawab.

Lapisan Interpretasi (1 fungsi, dependensi basis data + Sentry):

1. `lookupInterpretation(testId, totalScore, dimension?)`:
mengkueri tabel `result_interpretations` untuk pemetaan skor ke label, deskripsi, rekomendasi, dan tingkat keparahan. Fungsi ini *tidak murni* karena mengakses basis data dan mencatat peringatan ke Sentry pada kasus *interpretation miss*.

Tabel 2.3 menampilkan contoh skenario *unit test* yang diimplementasikan.

Tabel 2.3: Contoh Skenario *Unit Test* pada *Scoring Engine*

Fungsi	Kondisi Masukan	Hasil yang Diharapkan
<code>computeScore</code> (sumatif)	5 soal dengan nilai 4, 3, 5, 2, 4	<code>totalScore</code> = 18
<code>computeScore</code> (terbalik)	Soal <code>is_reversed=true</code> , skala 1–5, jawaban 2	Skor soal menjadi 4 (5 + 1 – 2)
<code>computeScore</code> (dimensional)	10 soal: 5 dimensi A, 5 dimensi B	<code>dimensionScores</code> terpisah per dimensi
<code>validateCompleteness</code>	Satu soal wajib belum dijawab	<code>valid</code> = false, <code>missingQuestionIds</code> berisi ID soal

Skenario-skenario pada Tabel 2.3 dipilih untuk mewakili setiap jalur komputasi yang mungkin dilalui sebuah jawaban, sehingga kesalahan pada satu jalur (misalnya pembalikan skor) akan terdeteksi tanpa tertutupi oleh jalur lain yang benar.

2.6.6 *End-to-End Testing* dengan Playwright

Playwright adalah *framework* otomasi *browser* yang dikembangkan oleh Microsoft untuk *E2E testing*. Tiga alur kritis yang direncanakan untuk *E2E testing* (sebagai pekerjaan lanjutan) adalah: (1) alur asesmen penuh dari katalog hingga halaman hasil; (2) alur autentikasi admin termasuk *redirect* otomatis; dan (3) alur manajemen

instrumen dari pembuatan hingga publikasi. Pengujian E2E ini masih berstatus implementasi awal dan belum ditinjau secara keseluruhan maupun di-*commit* ke repositori utama.

2.6.7 CI/CD dengan GitHub Actions

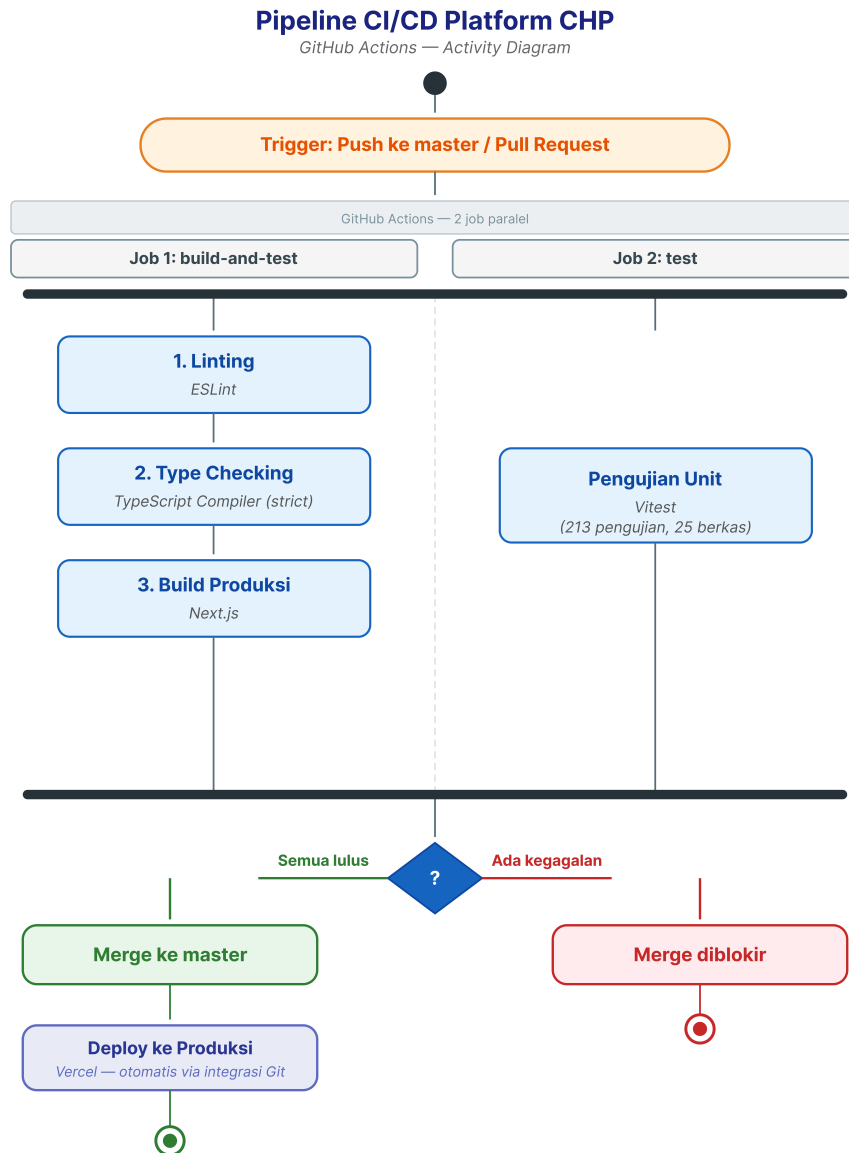
Continuous Integration/Continuous Deployment (CI/CD) menghilangkan proses integrasi dan *deployment* manual yang rawan kesalahan. Platform CHP mengimplementasikan *pipeline* CI/CD menggunakan GitHub Actions yang diaktifkan pada setiap *pull request* dan *push* ke cabang utama master.

Pipeline terdiri atas dua *job* yang berjalan secara paralel:

1. **Job 1:** *build-and-test*: menjalankan tiga *gate* secara berurutan:
 - (a) Pemeriksaan *linting* (ESLint): memastikan konsistensi gaya kode.
 - (b) Pemeriksaan *type checking* (TypeScript *Compiler* dengan *strict mode*): memastikan tidak ada kesalahan tipe.
 - (c) Proses *build* produksi (Next.js): memastikan aplikasi dapat dikompilasi untuk lingkungan produksi.
2. **Job 2:** *test*: menjalankan *unit test* (Vitest) secara paralel dengan *Job 1*, sehingga hasil pengujian tersedia lebih cepat tanpa menunggu proses *build* selesai.

Sebuah *pull request* tidak dapat di-*merge* ke cabang utama apabila salah satu dari kedua *job* tersebut gagal. Setelah perubahan ter-*merge* ke cabang master, Vercel melakukan *deployment* otomatis ke lingkungan produksi melalui integrasi Git-nya,

sehingga melengkapi tahap *Continuous Deployment*. Gambar 2.4 mengilustrasikan diagram alur *pipeline*.



Gambar 2.4: Diagram Alur *Pipeline* CI/CD Platform CHP (*Activity Diagram*)

Dengan *pipeline* ini, kualitas kode terjaga secara otomatis sebelum kode mencapai pengguna akhir. Lapisan pertahanan berikutnya berada pada data itu sendiri.

2.7 Keamanan dan Privasi Data

Platform yang menangani data psikologis sensitif memiliki kewajiban keamanan yang lebih tinggi dibandingkan aplikasi umum: hasil asesmen kesehatan mental adalah data pribadi yang bila bocor dapat merugikan pengisi secara langsung. Bagian ini meletakkan landasan teori keamanan yang digunakan platform, mulai dari standar risiko aplikasi (OWASP), mekanisme perlindungan tingkat aplikasi, strategi perlindungan data bertingkat, tanggung jawab keamanan infrastruktur pihak ketiga, hingga payung regulasi perlindungan data pribadi di Indonesia.

2.7.1 OWASP *Top Ten*

OWASP *Top Ten* menjadi standar acuan *de-facto* dalam komunitas keamanan perangkat lunak (OWASP Foundation 2026). Platform CHP mengimplementasikan mitigasi terhadap risiko yang paling relevan:

1. A01 *Broken Access Control*: Dimitigasi melalui RBAC lima tingkat dan pemeriksaan otorisasi pada setiap prosedur tRPC yang dilindungi.
2. A02 *Cryptographic Failures*: Data *in transit* dienkripsi via TLS/HTTPS. Data *at rest* dienkripsi oleh infrastruktur Neon PostgreSQL.
3. A03 *Injection*: Dicegah melalui Drizzle ORM yang menggunakan *parameterized queries*. Seluruh masukan pengguna divalidasi menggunakan skema Zod.
4. A07 *Identification and Authentication Failures*: Dimitigasi melalui JWT yang aman, *password hashing* menggunakan bcryptjs (*pure JavaScript*,

tanpa *native binding*) dengan *cost factor* 12, dan proteksi *brute force* melalui *rate limiting*.

2.7.2 *Rate Limiting* dan Perlindungan API

Platform CHP mengimplementasikan *rate limiting* pada *login* admin:

1. ***In-memory rate limiter*** untuk *login* admin: dua tingkat pembatasan, yaitu berbasis IP (5 kegagalan dalam 5 menit) dan berbasis akun (10 kegagalan yang mengunci akun hingga dibuka oleh *super_admin*). (Catatan: mekanisme *in-memory* ini pada hakikatnya tidak persisten dalam arsitektur *serverless* yang bersifat *stateless*. Migrasi ke layanan *cache* eksternal (seperti Redis) untuk ketahanan status lintas-sesi merupakan bagian dari mitigasi lanjutan).

2.7.3 Strategi Perlindungan Data Bertingkat

Tidak semua data dalam sebuah sistem membutuhkan (atau dapat menerima) mekanisme perlindungan yang sama. Prinsip yang menjadi acuan adalah bahwa tingkat perlindungan mengikuti fungsi data: mekanisme kriptografi dipilih berdasarkan apakah nilai asli data masih perlu dibaca kembali oleh sistem. Tiga tingkat perlindungan yang relevan bagi platform asesmen adalah sebagai berikut.

1. ***Hashing satu arah***: fungsi kriptografis (misalnya SHA-256) yang mengubah data menjadi sidik digital yang tidak dapat dibalikkan. Teknik ini tepat untuk data yang kegunaannya hanya perbandingan kesamaan (misalnya mendeteksi pengiriman ganda dari alamat IP yang sama), karena

sistem tidak pernah perlu mengetahui nilai aslinya.

2. **Enkripsi dua arah:** algoritma seperti AES-256-GCM yang menyandikan data namun dapat dipulihkan dengan kunci rahasia. Teknik ini tepat untuk data identitas yang jarang diakses tetapi sesekali harus dibaca kembali, misalnya alamat surel yang diperlukan saat pengiriman laporan.
3. **Penyimpanan terbaca dengan kontrol akses:** data yang dikonsumsi secara fungsional oleh aplikasi (ditampilkan di antarmuka, difilter, diurutkan, atau diekspor melalui kueri SQL) tidak dapat di-*hash* (nilainya hilang) dan menjadi tidak praktis bila dienkripsi per kolom (kueri WHERE, ORDER BY, dan agregasi tidak dapat bekerja pada *ciphertext*). Perlindungannya bertumpu pada lapisan lain: kontrol akses berbasis peran (RBAC), jejak audit, dan enkripsi *at rest* pada tingkat infrastruktur.

Di atas ketiga tingkat tersebut terdapat konsep pseudonimisasi yang diamanatkan semangat UU PDP: memisahkan data identitas dari data sensitif sedemikian rupa sehingga keduanya hanya terhubung melalui pengenal buatan (misalnya UUID sesi), bukan identitas langsung. Penerapan ketiga tingkat perlindungan ini pada skema basis data Platform CHP (kolom mana yang di-*hash*, mana yang dienkripsi, dan mana yang disimpan terbaca beserta justifikasinya) diuraikan sebagai keputusan perancangan pada Subbab 3.4.

2.7.4 Model Tanggung Jawab Bersama pada Infrastruktur *Cloud*

Ketika sistem berjalan di atas layanan *cloud* terkelola, keamanan menjadi tanggung jawab bersama (*shared responsibility model*): penyedia layanan bertanggung jawab

atas keamanan infrastruktur (fisik, jaringan, penyimpanan), sedangkan pemilik aplikasi bertanggung jawab atas keamanan pada tingkat aplikasi (kontrol akses, validasi masukan, pengelolaan kunci rahasia). Untuk lapisan infrastruktur, Platform CHP mengandalkan jaminan keamanan Neon selaku penyedia basis data terkelola: seluruh data dienkripsi *at rest* serta *in transit* melalui TLS, dan kepatuhan penyedia diverifikasi secara independen melalui sertifikasi SOC 2 Type 2 serta ISO 27001 (Neon 2026). Jaminan pihak ketiga ini melengkapi, bukan menggantikan, kontrol tingkat aplikasi yang diimplementasikan penulis.

Perlu diakui secara jujur bahwa tidak ada arsitektur yang dapat menjamin keamanan data secara absolut. Lanskap ancaman terus berevolusi, termasuk serangan yang semakin otomatis dan canggih di era AI. Posisi yang diambil dalam proyek ini adalah pertahanan berlapis (*defense in depth*): meminimalkan permukaan serangan pada tingkat aplikasi, menyerahkan keamanan infrastruktur kepada penyedia yang tersertifikasi, dan menyatakan risiko residual secara eksplisit sebagai batasan sistem (lih. Subbab 1.5).

2.7.5 Undang-Undang Perlindungan Data Pribadi

Undang-Undang Nomor 27 Tahun 2022 tentang Perlindungan Data Pribadi (UU PDP) mengamanatkan penerapan prinsip *privacy by design* dan minimisasi data (Rebong, Hambali, dan Timothy 2025). Platform CHP mengimplementasikan prinsip-prinsip UU PDP melalui empat mekanisme:

1. Sesi asesmen anonim tanpa memerlukan akun pengguna.
2. Penyamaran alamat IP: alamat IP di-*hash* satu arah sebelum disimpan.

3. Persetujuan eksplisit melalui modul `consents` yang merekam penerimaan *Terms of Service*, *Research Opt-In*, dan *Marketing Opt-In* secara terpadu melalui satu kotak centang pada antarmuka pengguna (*frontend*). Meski secara visual disatukan untuk kenyamanan pengguna, nilai-nilai persetujuan ini direkam secara terpisah dan baku (*hardcoded*) oleh sistem di sisi peladen (*backend*) untuk menjamin kepatuhan audit.
4. Enkripsi surel peserta menggunakan AES-256-GCM dalam tabel `report_requests` yang terisolasi.

BAB 3

ANALISA DAN PERANCANGAN

Bab ini memaparkan proses analisis kebutuhan dan perancangan sistem yang menjadi landasan pembangunan Platform Asesmen Digital CHP. Sesuai dengan proses rekayasa perangkat lunak yang dikemukakan Sommerville (2016), aktivitas spesifikasi dan perancangan dilaksanakan secara iteratif. Analisis kebutuhan mengidentifikasi apa yang harus dilakukan sistem, sedangkan perancangan mendefinisikan bagaimana sistem akan memenuhi kebutuhan tersebut.

3.1 Analisis Kebutuhan Fungsional

Analisis kebutuhan fungsional mengidentifikasi fungsi-fungsi spesifik yang harus disediakan oleh sistem. Berdasarkan hasil diskusi dengan Ibu Astin selaku pemilik proyek serta observasi terhadap sistem CHP terdahulu, empat aktor utama teridentifikasi: (1) Peserta Anonim; (2) Pengguna Terdaftar; (3) Administrator; dan (4) Super Admin.

3.1.1 Kebutuhan Pengguna

Tabel 3.1 menyajikan kebutuhan dari perspektif setiap aktor pengguna.

Tabel 3.1: Kebutuhan Pengguna Platform CHP

Aktor	Kategori	Kebutuhan
Peserta Anonim	Asesmen	Mengambil asesmen psikologi tanpa membuat akun, melihat hasil instan, meminta laporan melalui surel
Pengguna Terdaftar	Akun & Riwayat	Semua fitur peserta anonim, ditambah klaim sesi anonim ke akun, riwayat asesmen, dan manajemen profil
Administrator	Manajemen	Mengelola instrumen tes (CRUD), melihat statistik dan hasil, mengelola permintaan laporan, mengeksport data riset
Super Admin	Keamanan	Semua fitur administrator, ditambah manajemen akun admin, aktivasi/deaktivasi pengguna, dan jejak audit

Kebutuhan dari sisi pengguna pada Tabel 3.1 tersebut kemudian diterjemahkan menjadi kebutuhan teknis pada sisi perangkat lunak dan perangkat keras.

3.1.2 Kebutuhan Perangkat Lunak

Tabel 3.2 menyajikan kebutuhan perangkat lunak untuk pengembangan dan produksi.

Selain perangkat lunak pada Tabel 3.2, sistem juga bergantung pada kebutuhan

Tabel 3.2: Kebutuhan Perangkat Lunak

Konteks	Perangkat Lunak	Versi/Spesifikasi
Pengembangan	Node.js	v22 (LTS)
Pengembangan	pnpm	v9 (<i>package manager</i>)
Pengembangan	TypeScript	v6.0.3 (<code>strict: true</code>)
Pengembangan	Git	Kontrol versi terdistribusi
<i>Framework</i>	Next.js	v16.2.4 (<i>App Router</i>)
<i>Framework</i>	React	v19.2.5 (dengan <i>React Compiler</i>)
<i>Framework</i>	tRPC	v11.16.0
ORM	Drizzle ORM	v1.0.0-beta.20
Basis Data	PostgreSQL	v17 (melalui Neon <i>serverless</i>)
Autentikasi	Auth.js (NextAuth)	v5 (next-auth 5.0.0-beta.30)
Autentikasi Admin	jose	v6.2.3 (JWT mandiri)
Pengujian	Vitest	v4.1.5
Pengujian	Playwright	v1.59.1
Validasi	Zod	v4.3.6
<i>Monitoring</i>	Sentry	v10.50.0
Surel	Resend	v6.12.2
PDF	@react-pdf/renderer	v4.5.1
Kontainerisasi	Docker	Engine v24+ dengan Compose v2 (opsional, untuk instalasi mandiri)

perangkat keras berikut.

3.1.3 Kebutuhan Perangkat Keras

Tabel 3.3 menyajikan kebutuhan perangkat keras.

Tabel 3.3: Kebutuhan Perangkat Keras

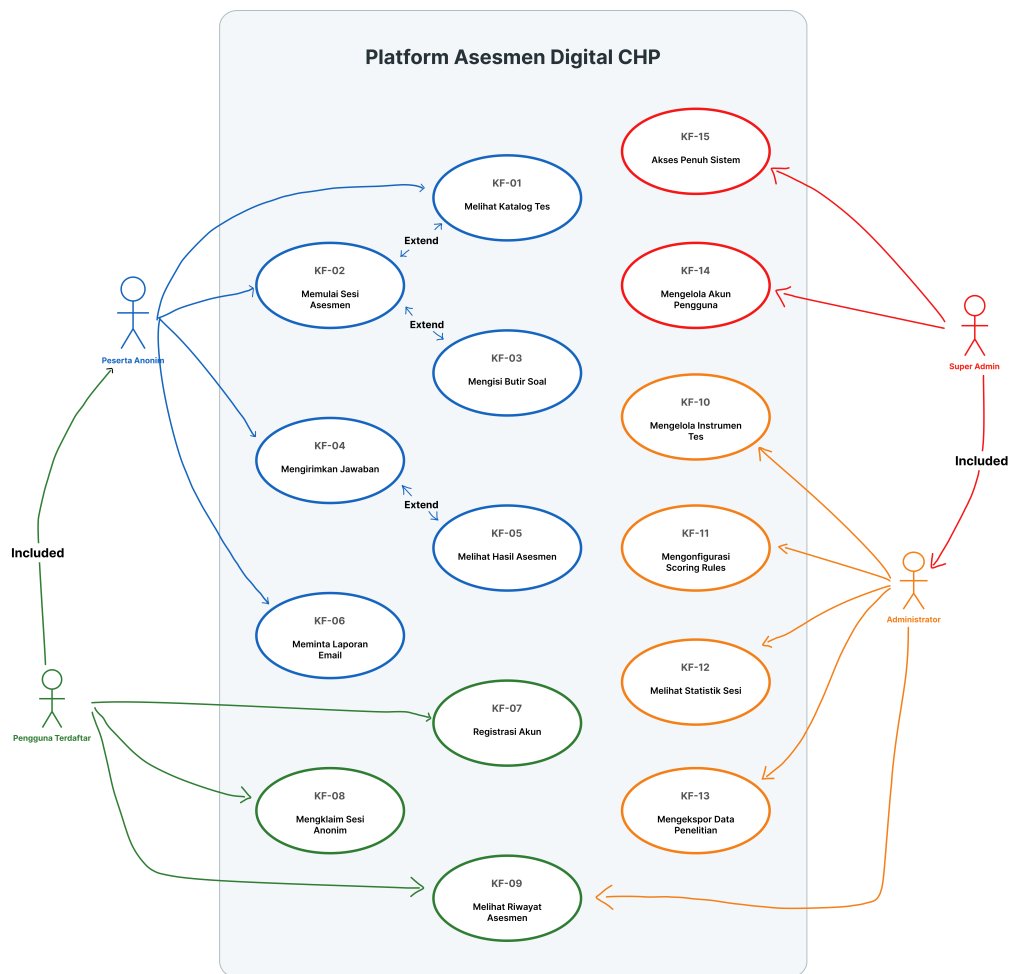
Konteks	Komponen	Spesifikasi
Server	Komputasi	Vercel <i>serverless</i> (<i>auto-scaling</i>)
Server	Basis Data	Neon PostgreSQL terkelola (<i>scale-to-zero</i>)
Klien	Peramban	Peramban modern (Chrome, Firefox, Safari, Edge)
Klien	Layar	Lebar minimum 320 px (responsif hingga 2560 px)
Pengembangan	Komputer	RAM $\geq$ 8 GB, penyimpanan $\geq$ 256 GB SSD
Server alternatif	<i>On-premise</i>	CPU 2 <i>core</i> , RAM 4 GB, SSD 20 GB (rincian pada Subbab 4.10.4)

Baris terakhir pada kedua tabel di atas mencerminkan bahwa platform dirancang tidak terikat pada satu model *deployment*: selain jalur *cloud serverless* yang digu-

nakan saat ini, platform dapat diinstalasi secara mandiri pada server milik institusi sebagaimana diuraikan pada Subbab 4.10.

3.1.4 Diagram Use Case

Gambar 3.1 menyajikan diagram *use case* yang mengilustrasikan interaksi empat aktor dengan sistem.



Gambar 3.1: Diagram *Use Case* Platform Asesmen Digital CHP

Setiap interaksi pada Gambar 3.1 diterjemahkan menjadi satu atau lebih kebutuhan fungsional berkode KF, sebagaimana dirinci pada Tabel 3.4 berikut.

Tabel 3.4: Kebutuhan Fungsional Platform CHP

ID	Deskripsi	Aktor	Prioritas
KF-01	Melihat katalog instrumen asesmen yang tersedia	Peserta Anonim	Tinggi
KF-02	Memulai sesi asesmen baru dengan persetujuan eksplisit	Peserta Anonim	Tinggi
KF-03	Mengisi butir soal secara bertahap dengan <i>auto-save</i>	Peserta Anonim	Tinggi
KF-04	Mengirimkan jawaban dan menerima skor instan	Peserta Anonim	Tinggi
KF-05	Melihat hasil asesmen (skor total, dimensi, interpretasi)	Peserta Anonim	Tinggi
KF-06	Meminta pengiriman laporan melalui surel	Peserta Anonim	Sedang
KF-07	Registrasi akun dengan surel dan kata sandi	Pengguna Terdaftar	Tinggi
KF-08	Mengklaim sesi anonim ke akun terdaftar	Pengguna Terdaftar	Tinggi
KF-09	Melihat riwayat asesmen	Pengguna Terdaftar	Sedang
KF-10	Mengelola instrumen tes melalui CRUD	Administrator	Tinggi

ID	Deskripsi	Aktor	Prioritas
KF-11	Mengonfigurasi aturan penilaian per instrumen	Administrator	Tinggi
KF-12	Melihat statistik sesi dan hasil asesmen	Administrator	Sedang
KF-13	Mengekspor data penelitian	Administrator	Sedang
KF-14	Mengelola peran dan akses pengguna (<i>Account Management</i>)	Super Admin	Tinggi
KF-15	Memiliki akses penuh dan <i>override</i> terhadap data dan konfigurasi sistem	Super Admin	Tinggi

Di luar fungsi-fungsi pada Tabel 3.4 tersebut, sistem juga harus memenuhi sejumlah kualitas non-fungsional agar layak disebut berkualitas produksi.

3.2 Analisis Kebutuhan Non-Fungsional

Tabel 3.5 menyajikan kebutuhan non-fungsional Platform CHP. Catatan: Target terukur yang tercantum merupakan target desain arsitektur; pengukuran performa secara formal (beban konkuren, *uptime*, dsb.) merupakan lingkup pekerjaan lanjutan.

Tabel 3.5: Kebutuhan Non-Fungsional Platform CHP

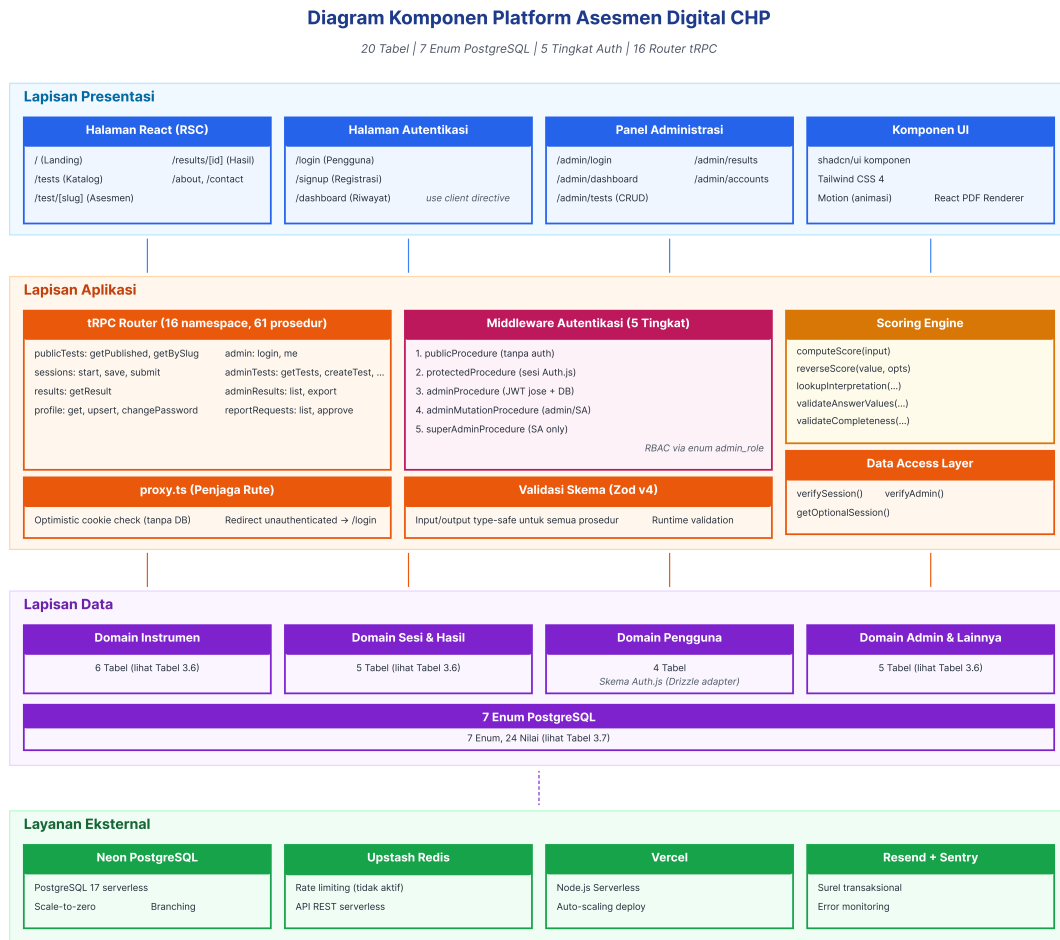
Kategori	Parameter	Target Terukur	Strategi Implementasi
Performa	Waktu muat halaman	<i>Lighthouse</i> ≥ 90	RSC untuk mengurangi <i>bundle</i> JavaScript

Kategori	Parameter	Target Terukur	Strategi Implementasi
Performa	Waktu respons API	≤ 500 ms (P95)	Vercel <i>Serverless Functions</i>
Keamanan	OWASP	Mitigasi A01, A02, A03, A07	<i>Parameterized queries</i> , JWT, RBAC, <i>rate limiting</i>
Keamanan	Privasi data	Kepatuhan UU PDP	Sesi anonim, IP di-hash, surel dienkripsi AES-256-GCM
Skalabilitas	Beban konku- ren	≥ 100 sesi simultan	Arsitektur <i>serverless</i>
Kegunaan	Aksesibilitas	<i>Lighthouse</i> ≥ 95	Semantik HTML5, kontras WCAG 2.1 AA
Kegunaan	Responsivitas	320 px–2560 px	Desain <i>mobile-first</i>
Keandalan	Ketersediaan	<i>Uptime</i> $\geq 99.9\%$	Infrastruktur terkelola Vercel + Neon
Keandalan	Integritas data	Nol kehilangan data	<i>Idempotency guard</i> , penjaga status sesi
Pemeliharaan	Kualitas kode	Nol <i>error</i> ESLint + TypeScript <i>strict</i>	<i>Pipeline</i> CI/CD dengan <i>quality gates</i>

Kebutuhan fungsional dan non-fungsional pada kedua tabel di atas menjadi acuan bagi seluruh keputusan arsitektur yang diuraikan pada bagian berikut.

3.3 Perancangan Arsitektur Sistem

Gambar 3.2 menyajikan diagram komponen yang memperluas Gambar 2.1 dengan menunjukkan modul-modul spesifik.



Gambar 3.2: Diagram Komponen Platform Asesmen Digital CHP

Lapisan aplikasi terdiri atas empat modul utama:

1. *Router tRPC (16 namespace)* yang mengekspos 61 prosedur API.
2. Inti penilaian murni (`computeScore`, `reverseScore`, `validateAnswerValues`, `validateCompleteness`) yang mengandung logika komputasi skor deterministik tanpa dependensi eksternal; serta lapisan in-

terpretasi (`lookupInterpretation`) yang mengkueri basis data untuk pemetaan skor ke label.

3. *Middleware* autentikasi yang menerapkan lima tingkat kontrol akses: `publicProcedure`, `protectedProcedure`, `adminProcedure`, `adminMutationProcedure`, dan `superAdminProcedure`.
4. *Data Access Layer* (DAL) yang menyediakan fungsi `verifySession`, `verifyAdmin`, dan `getOptionalSession`.

3.3.1 Batas Klien-Server

Modul `proxy.ts` (pengganti `middleware.ts` pada Next.js 16) berfungsi sebagai penjaga rute optimistis yang membaca *cookie* sesi tanpa mengakses basis data. Pemeriksaan otorisasi definitif dilakukan oleh DAL di lapisan aplikasi, sehingga keamanan bergantung pada dua lapis pertahanan yang saling menguatkan.

3.4 Perancangan Basis Data

Skema basis data Platform CHP dirancang dengan tiga prinsip utama: (1) fleksibilitas untuk beragam instrumen psikologi; (2) pemisahan data kesehatan dari data identitas pribadi; dan (3) reproduktibilitas ilmiah melalui penyimpanan versi instrumen dan algoritma.

3.4.1 Model Domain

Skema basis data diorganisasi ke dalam enam domain:

1. **Domain Instrumen Asesmen** (6 tabel): mendefinisikan struktur instrumen psikometri.
2. **Domain Sesi dan Hasil** (5 tabel): mencatat pelaksanaan asesmen.
3. **Domain Pengguna** (4 tabel): menyediakan infrastruktur identitas dan profil.
4. **Domain Admin dan Audit** (2 tabel): mendefinisikan pengguna administratif dan jejak audit.
5. **Domain Permintaan dan Surel** (2 tabel): mengelola permintaan laporan dan PII.
6. **Domain Infrastruktur Autentikasi** (1 tabel): *token* verifikasi.

3.4.2 Skema Tabel

Tabel domain aplikasi menggunakan UUID v4 sebagai kunci primer. Tabel adapter Auth.js (*verification_tokens*, *accounts*, *auth_sessions*) mengikuti kontrak kunci yang ditentukan oleh Auth.js, khususnya *verification_tokens* yang menggunakan kunci primer komposit (*identifier*, *token*). Tabel 3.6 menyajikan inventaris lengkap 20 tabel.

Tabel 3.6: Inventaris Tabel Basis Data Platform CHP

Domain	Tabel	Kolom Kunci	Tujuan
Instrumen	tests	id, slug, title,	Metadata instrumen
		version	psikometri

Domain	Tabel	Kolom Kunci	Tujuan
Instrumen	questions	id, test_id, type, dimension	Butir soal dengan dimensi dan pembobotan
Instrumen	options	id, question_id, label, value	Opsi jawaban per butir soal
Instrumen	scoring_rules	id, test_id, algorithm, config	<i>Schema placeholder</i> (tidak digunakan)
Instrumen	result_interpretations	id, test_id, dimension, min/max_score	Pemetaan skor ke label interpretasi
Instrumen	test_citations	id, test_id, type, citation, doi	Sitasi akademik per instrumen
Sesi	test_sessions	id, test_id, status, claim_token	Sesi asesmen dengan mekanisme klaim
Sesi	answers	id, session_id, question_id, value	Respons mentah yang <i>immutable</i>

Domain	Tabel	Kolom Kunci	Tujuan
Sesi	results	id, session_id, total_score, dimension_scores, raw_scores, computed_scores, result_label, scoring_version	Skor terhitung terpisah dari respons mentah; scoring_version melacak versi algoritma
Sesi	session_demo_graphics	session_id, name, sex, age, province	Data demografis peserta per sesi
Sesi	consents	id, session_id, tos_accepted, consent_version	Persetujuan eksplisit sesuai UU PDP
Pengguna	users	id, email, password_hash, role	Identitas pengguna terdaftar
Pengguna	user_profiles	user_id, display_name, sex, age	Profil pengguna yang diperluas

Domain	Tabel	Kolom Kunci	Tujuan
Pengguna	accounts	id, user_id, provider	Tautan akun OAuth (infrastruktur pasif, berbasis JWT)
Pengguna	auth_sessions	id, session_token, user_id, expires	Sesi autentikasi basis data (infrastruktur pasif)
Admin	admin_users	id, email, role, is_active	Pengguna administratif dengan peran
Admin	audit_logs	id, admin_user_id, action, entity_type	Jejak audit <i>append-only</i>
Permintaan	report_requests	id, session_id, status, requester_type	Permintaan laporan dengan alur persetujuan
Permintaan	guest_leads	id, session_id, encrypted_email	PII terenkripsi AES-256-GCM
Auth	verification_tokens	identifier, token, expires	<i>Token</i> verifikasi sekali pakai

Selain 20 tabel, skema mendefinisikan tujuh *enum* PostgreSQL yang memastikan integritas data. Tabel 3.7 mencantumkan definisinya.

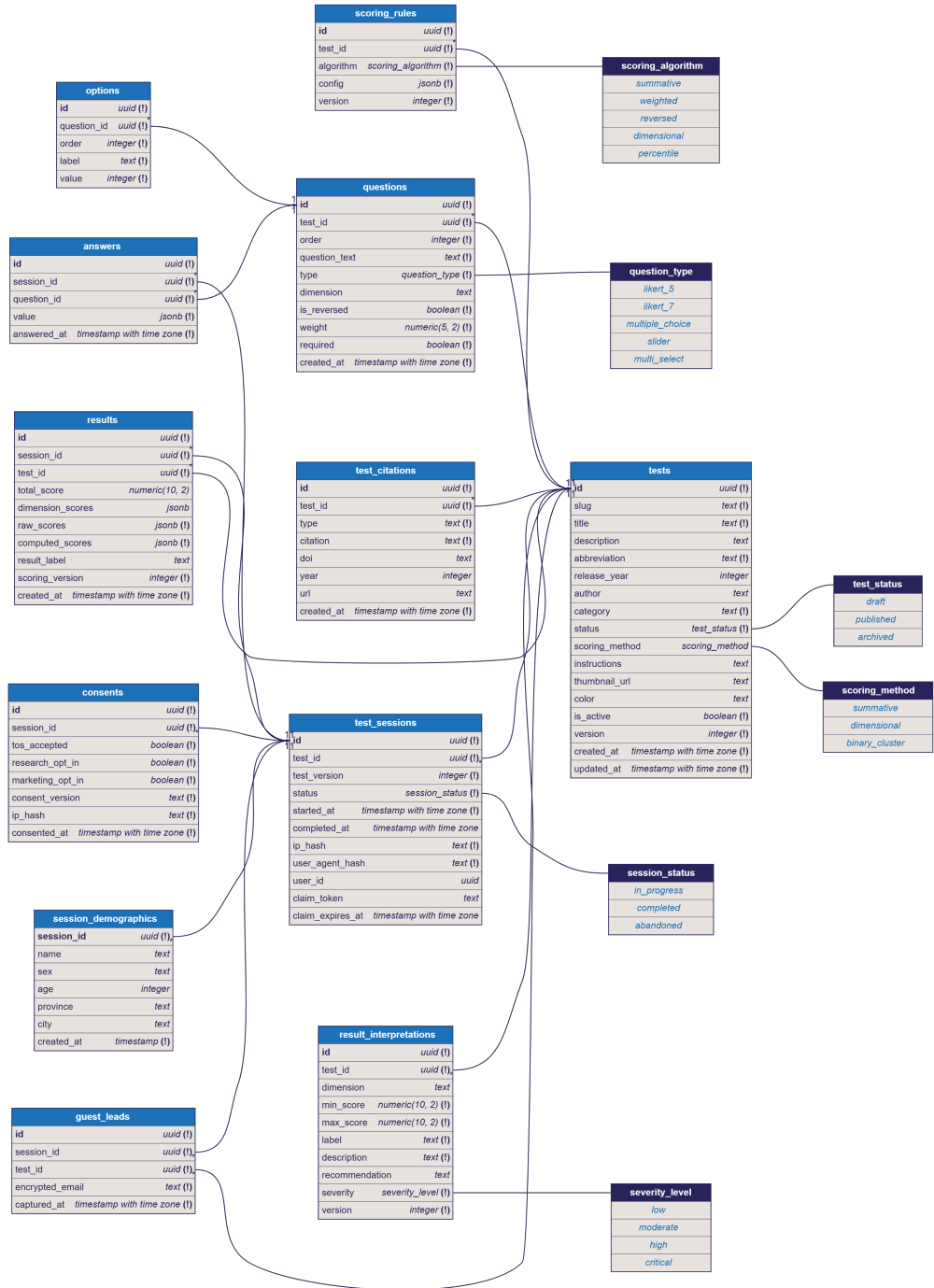
Tabel 3.7: Definisi *Enum* PostgreSQL

Nama Enum	Nilai yang Dii- zinkan	Digunakan Pada
<code>test_status</code>	<code>draft,</code> <code>published,</code> <code>archived</code>	<code>tests.status</code>
<code>scoring_method</code>	<code>summative,</code> <code>dimensional,</code> <code>binary_cluster</code>	<code>tests.scoring_method</code>
<code>question_type</code>	<code>likert_5,</code> <code>likert_7,</code> <code>multiple_choice,</code> <code>slider,</code> <code>multi_select</code>	<code>questions.type</code>
<code>scoring_algorithm</code>	<code>summative,</code> <code>weighted,</code> <code>reversed,</code> <code>dimensional</code>	<code>scoring_rules.algorithm</code>
<code>severity_level</code>	<code>low,</code> <code>moderate,</code> <code>high,</code> <code>critical</code>	<code>result_interpretations.severity</code>
<code>session_status</code>	<code>in_progress,</code> <code>completed,</code> <code>abandoned</code>	<code>test_sessions.status</code>
<code>admin_role</code>	<code>super_admin,</code> <code>admin,</code> <code>psychiatrist,</code> <code>researcher</code>	<code>admin_users.role</code>

Setelah struktur tabel dan *enum* ditetapkan, langkah perancangan berikutnya adalah mendefinisikan bagaimana entitas-entitas tersebut saling berelasi.

3.4.3 Relasi Antar-Entitas

Gambar 3.3 di atas menampilkan klaster inti yang menangani siklus hidup asesmen; klaster kedua yang menangani identitas dan administrasi ditampilkan terpisah pada



Gambar 3.3: Diagram *Entity-Relationship* Klaster Inti (Asesmen & Pengguna).

Gambar 3.4 berikut agar kedua diagram tetap terbaca pada ukuran kertas laporan.

Kedua klaster dihubungkan oleh 4 *foreign key* lintas-klaster: `test_sessions.test_id`, `test_sessions.user_id`, `results.test_id`, dan `report_requests.session_id`.

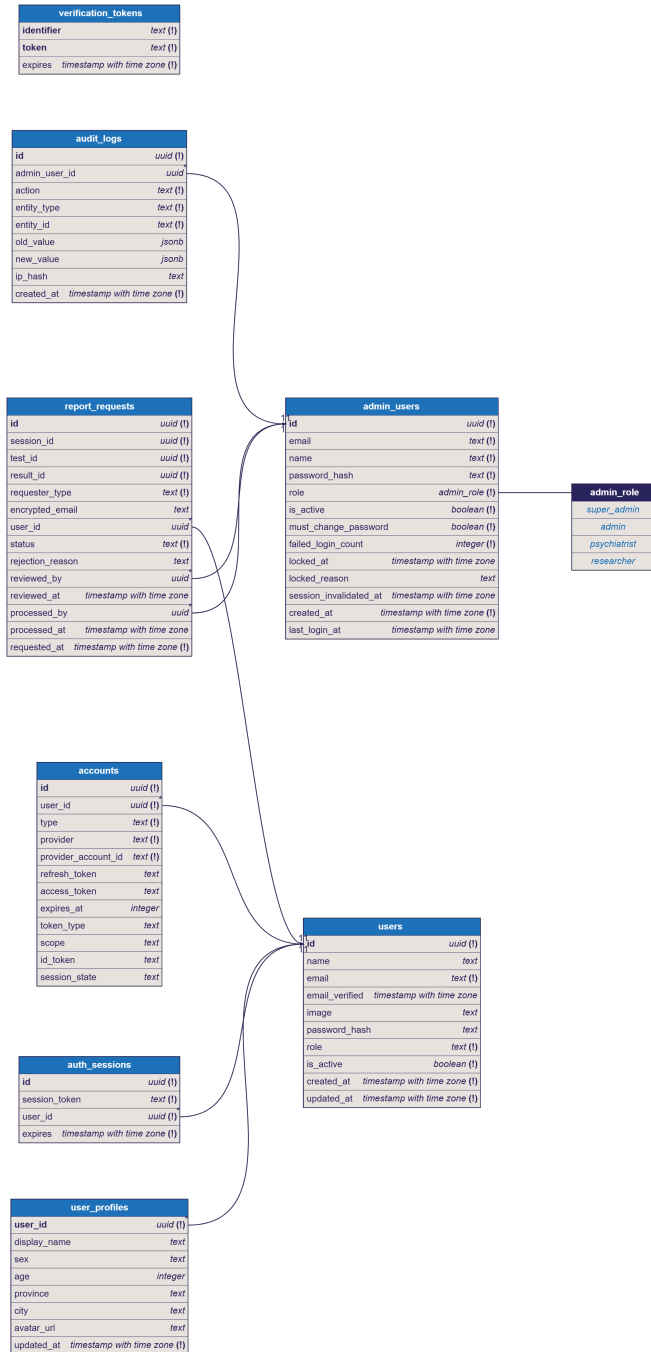
Relasi utama antar-entitas:

1. `tests` → `questions` → `options`: pola satu-ke-banyak bertingkat dengan `ON DELETE CASCADE`.
2. `test_sessions` → `answers`: *constraint* unik komposit (`session_id`, `question_id`).
3. `test_sessions` → `results`: relasi satu-ke-satu melalui *constraint* `UNIQUE` pada `session_id`.
4. `test_sessions` → `users`: *foreign key* opsional (`user_id nullable`) dengan `ON DELETE SET NULL`.
5. `admin_users` → `audit_logs`: `ON DELETE SET NULL` agar penghapusan akun admin tidak menghapus rekam jejak.

3.4.4 Keputusan Perancangan Kritis

Empat keputusan perancangan data dengan dampak arsitektur signifikan:

1. **Pemisahan** `answers` **dan** `results`: memungkinkan *re-scoring* tanpa kehilangan data historis.
2. **JSONB untuk nilai polimorfik**: kolom `answers.value` mengakomodasi format jawaban beragam (Likert, pilihan ganda, *slider*).



Gambar 3.4: Diagram *Entity-Relationship* Klaster Administratif.

3. **Claim token:** UUID v4 dengan masa berlaku 72 jam untuk transisi anonim-ke-terdaftar; sekali pakai dan dinihilkan setelah diklaim.
4. **Perlindungan data selektif bertingkat:** setiap kolom dilindungi sesuai fungsinya, mengikuti kerangka teori pada Subbab 2.7.3, sebagaimana dirinci di bawah.

Keputusan keempat perlu diuraikan lebih jauh karena sekilas dapat terlihat sebagai inkonsistensi: pada klaster sesi yang sama, kolom `ip_hash` dan `user_agent_hash` di tabel `test_sessions` disimpan dalam bentuk *hash*, sedangkan tabel `session_demographics` yang berelasi satu-ke-satu dengannya menyimpan nama, jenis kelamin, usia, dan domisili dalam bentuk terbaca. Perbedaan ini bukan kelalaian, melainkan konsekuensi dari fungsi masing-masing data:

1. **Hashing satu arah** untuk alamat IP dan *user agent*: kedua nilai ini hanya digunakan untuk mendeteksi pengiriman ganda dan penyalahgunaan. Sistem cukup membandingkan kesamaan *hash* dan tidak pernah perlu membaca nilai aslinya, sehingga bentuk yang tidak dapat dibalikkan justru paling aman.
2. **Enkripsi dua arah (AES-256-GCM)** untuk alamat surel: surel tidak ditampilkan di antarmuka mana pun, tetapi harus dapat dipulihkan sesaat ketika laporan hasil dikirimkan. Surel peserta dienkripsi pada tabel `report_requests`; tabel `guest_leads` menerapkan pola enkripsi yang sama (secara operasional tabel ini merupakan infrastruktur skema untuk fitur pengumpulan prospek di masa mendatang dan saat ini belum

menampung data aktif).

3. **Penyimpanan terbaca dengan kontrol akses** untuk data demografis: nama, usia, dan domisili dikonsumsi langsung oleh fungsionalitas inti: ditampilkan pada laporan hasil individual, serta dicari (LIKE), difilter, diurutkan, dan diekspor pada panel admin melalui kueri SQL. Enkripsi per kolom akan mematahkan seluruh kemampuan kueri tersebut. Sebagai kompensasi, akses ke data ini dibatasi berlapis: hanya dapat dibaca melalui prosedur ber-*middleware* `adminProcedure` (RBAC), setiap mutasi administratif terekam pada `audit_logs`, data demografis dibekukan setelah sesi selesai, dan penyimpanan fisiknya dienkripsi *at rest* oleh infrastruktur Neon.

Dengan strategi ini, identitas peserta tidak pernah berdampingan dengan data kesehatan secara langsung: tabel `answers` dan `results` tidak memuat satu pun kolom identitas, dan keduanya hanya terhubung ke demografi melalui pengenalan pseudonim `session_id`. Batasan yang tersisa dari pendekatan ini (demografi tetap terbaca bagi pemegang akses basis data langsung) dinyatakan secara eksplisit pada Subbab 1.5, dan pseudonimisasi lanjutan dicatat sebagai saran pengembangan pada Subbab 5.2.

3.5 Perancangan Antarmuka Pemrograman Aplikasi

Platform CHP menggunakan tRPC sebagai protokol komunikasi. Seluruh 61 prosedur API diorganisasi ke dalam 16 *router namespace*. Tabel 3.8 hingga Tabel 3.10 menyajikan inventaris prosedur yang dikelompokkan berdasarkan area fungsional.

Tabel 3.8: Prosedur API Publik (Alur Asesmen)

Prosedur	Router	Tipe	Auth	Fungsi
<code>getPublishedTests</code>	<code>publicTests</code>	Q	Publik	Ambil daftar tes aktif
<code>getTestBySlug</code>	<code>publicTests</code>	Q	Publik	Ambil tes berdasarkan <i>slug</i>
<code>startSession</code>	<code>sessions</code>	M	Publik	Mulai sesi, simpan <i>consent</i> + <i>claim token</i>
<code>saveDemographics</code>	<code>sessions</code>	M	Publik	Simpan data demografis
<code>saveProgress</code>	<code>sessions</code>	M	Publik	Simpan jawaban (<i>batch upsert</i>)
<code>submitAssessment</code>	<code>sessions</code>	M	Publik	Validasi, hitung skor, simpan hasil
<code>getResult</code>	<code>results</code>	Q	Publik	Ambil hasil asesmen
<code>requestEmailReport</code>	<code>sessions</code>	M	Publik	Buat permintaan laporan surel

Selain prosedur publik pada Tabel 3.8, pengguna yang telah mendaftar memperoleh akses ke prosedur tambahan berikut.

Tabel 3.9: Prosedur API Pengguna Terdaftar

Prosedur	Router	Tipe	Auth	Fungsi
<code>claimSession</code>	<code>sessions</code>	M	Protected	Klaim sesi anonim ke akun
<code>get</code>	<code>profile</code>	Q	Protected	Ambil profil pengguna
<code>upsert</code>	<code>profile</code>	M	Protected	Simpan/perbarui profil

Prosedur	Router	Tipe	Auth	Fungsi
changePassword	profile	M	Protected	Ganti kata sandi

Pada sisi administratif, prosedur berikut disediakan khusus bagi tim pengelola platform.

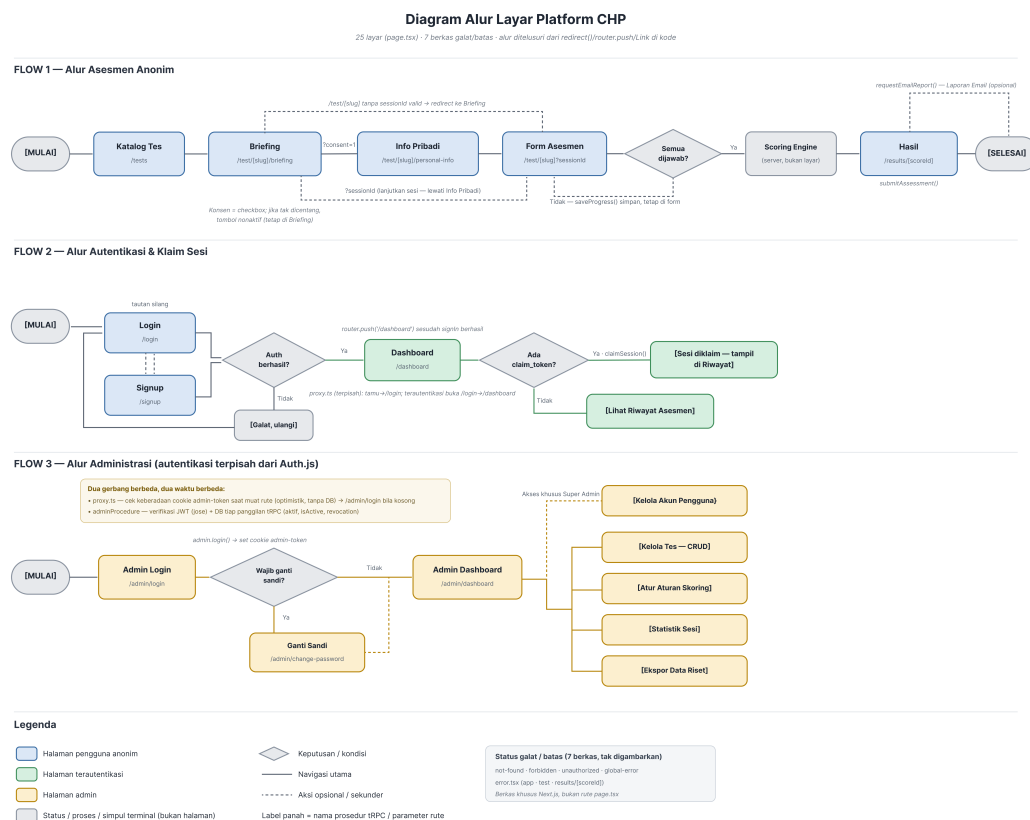
Tabel 3.10: Prosedur API Panel Administrasi

Prosedur	Router	Tipe	Auth	Fungsi
login	admin	M	Publik	<i>Login</i> admin, terbitkan JWT
me	admin	Q	Admin	Profil admin aktif
stats	adminDashboard	Q	Admin	Statistik platform
list	adminResults	Q	Admin	Tabel hasil per instrumen
export	adminResults	Q	Admin	Ekspor data (maks 5000 baris)
list	reportRequests	Q	Admin	Daftar permintaan laporan
approve	reportRequests	M	Admin	<i>Generate</i> PDF + kirim surel
getTests	adminTests	Q	Admin	Daftar instrumen
createTest	adminTests	M	AdminMut	Buat tes baru
createAdmin	adminAccounts	M	SuperAdmin	Buat akun admin

Setiap prosedur memanfaatkan lima tingkat *middleware* autentikasi: `publicProcedure` (tanpa autentikasi), `protectedProcedure` (sesi Auth.js), `adminProcedure` (JWT admin via jose dengan validasi basis data), `adminMutationProcedure` (peran `super_admin` atau `admin`), dan `superAdminProcedure` (khusus `super_admin`).

3.6 Perancangan Alur Antarmuka Pengguna

Perancangan komponen antarmuka pengguna secara visual merupakan tanggung jawab anggota tim lain. Namun, perancangan alur layar (*screen flow*) merupakan bagian integral dari arsitektur sistem.



Gambar 3.5: Diagram Alur Layar Platform CHP

Tiga alur utama:

1. **Alur asesmen anonim:** `/tests` → `/test/[slug]` → `/results/[scoreId]`. Seluruh alur berjalan tanpa memerlukan akun.
2. **Alur autentikasi dan klaim sesi:** `/signup` atau `/login` → `/dashboard`. Penjaga rute `proxy.ts` memastikan halaman terlindungi hanya diakses oleh pengguna terautentikasi.
3. **Alur administrasi:** `/admin/login` → `/admin/dashboard` → halaman manajemen instrumen, hasil, laporan, dan akun.

BAB 4

IMPLEMENTASI DAN PENGUJIAN

Bab ini memaparkan implementasi sistem berdasarkan perancangan yang telah diuraikan pada Bab 3. Pembahasan mencakup lingkungan pengembangan, implementasi basis data, *scoring engine*, API tRPC, sistem autentikasi, panel administrasi, generasi PDF dan pengiriman surel, pengujian unit, *pipeline* CI/CD, serta *deployment* ke lingkungan produksi beserta opsi instalasi mandiri (*on-premise*) pada server institusi.

4.1 Lingkungan Pengembangan

Pengembangan Platform CHP dilakukan pada lingkungan lokal dengan spesifikasi perangkat lunak yang tercantum pada Tabel 3.2. Manajer paket yang digunakan adalah pnpm versi 9, yang dipilih karena efisiensi penyimpanannya melalui mekanisme *content-addressable store* yang menghindari duplikasi dependensi antar proyek.

Konfigurasi TypeScript menggunakan `mode strict: true` dengan opsi tambahan `noUncheckedIndexedAccess`, `noUnusedLocals`, dan `noUnusedParameters` untuk memaksimalkan deteksi kesalahan pada waktu kompilasi. Tiga *path alias* dikonfigurasi untuk menyederhanakan impor modul: `@/*` untuk direktori `src/`, `@/server/*` untuk `src/server/`, dan `@/components/*` untuk `src/components/`.

Seluruh *environment variable* yang diperlukan didokumentasikan dalam *file* `.env.example` yang berisi 14 variabel konfigurasi aktif dan 2 variabel opsional untuk *bootstrap* akun admin awal (dikomentari secara *default*), mencakup koneksi basis

data (DATABASE_URL), kunci autentikasi (AUTH_SECRET, ADMIN_JWT_SECRET), konfigurasi layanan pihak ketiga (Sentry, Resend), dan kunci enkripsi (ENCRYPTION_KEY sepanjang 64 karakter heksadesimal untuk AES-256-GCM).

4.2 Implementasi Basis Data

Skema basis data yang dirancang pada Subbab 3.4 diimplementasikan menggunakan Drizzle ORM versi 1.0.0-beta.20 dengan *driver* Neon HTTP (@neondatabase/serverless). Definisi skema diorganisasi ke dalam 14 *file* TypeScript di dalam direktori `src/server/schema/`, dengan setiap *file* bertanggung jawab atas satu domain.

Seluruh 20 tabel dan 7 *enum* PostgreSQL berhasil di-*deploy* ke lingkungan produksi Neon PostgreSQL. Migrasi skema dikelola melalui Drizzle Kit (`drizzle-kit`) yang menghasilkan *file* SQL migrasi secara otomatis berdasarkan perubahan pada definisi skema TypeScript. Pendekatan ini memastikan bahwa setiap perubahan skema terdokumentasi dan dapat direproduksi.

Relasi antar-tabel diimplementasikan menggunakan `defineRelations()` dari Drizzle ORM v2, yang memisahkan definisi relasi dari definisi kolom. Pola ini memungkinkan kueri relasional (*relational queries*) yang menghasilkan SQL JOIN secara otomatis tanpa memerlukan penulisan kueri manual.

Data awal (*seed data*) untuk empat instrumen psikometri, yaitu PSS-10, GPIUS-2, SRS, dan SRQ-29, disiapkan melalui skrip *seeding* yang memasukkan definisi tes, butir soal, opsi jawaban, dan rentang interpretasi (yang diturunkan dari manual instrumen asli atau batas proporsional empiris sesuai rujukan ahli materi). Skrip

ini memastikan lingkungan pengembangan baru dapat langsung memiliki data yang cukup untuk pengujian.

4.3 Implementasi *Scoring Engine*

Scoring engine diimplementasikan sebagai *pipeline* fungsi murni (*pure functions*) dalam tiga *file* di direktori `src/server/scoring/`. Desain ini menjamin determinisme: untuk masukan yang sama, keluaran selalu identik, tanpa efek samping (*side effects*) atau ketergantungan pada status eksternal.

4.3.1 Fungsi `reverseScore`

Fungsi `reverseScore(rawValue, options)` mengimplementasikan pembalikan skor secara agnostik skala menggunakan rumus:

$$\text{reversedScore} = (\text{scaleMax} - \text{rawValue}) + \text{scaleMin} \quad (4.1)$$

di mana `scaleMax` dan `scaleMin` diturunkan secara otomatis dari larik opsi yang diberikan. Pendekatan ini menghilangkan kebutuhan untuk mengonfigurasi batas skala secara manual, sehingga fungsi dapat digunakan untuk skala Likert dengan rentang apapun.

4.3.2 Fungsi `computeScore`

Fungsi `computeScore(input)` adalah inti dari *scoring engine*. Fungsi ini menerima masukan berupa jawaban peserta beserta metadata soal, dan menghasilkan objek hasil yang mencakup:

1. Skor total (`totalScore`): penjumlahan seluruh skor butir soal setelah pembalikan dan pembobotan. Formula dasarnya adalah $S_{total} = \sum_{i=1}^n (v_i \times w_i)$, di mana v_i adalah nilai jawaban dan w_i adalah bobot soal.
2. Skor per dimensi (`dimensionScores`): agregasi dimensional dengan menjumlahkan butir soal pada masing-masing dimensi secara terpisah.
3. Deteksi klaster biner (*binary-cluster*): pada instrumen skrining seperti SRQ-29, komputasi dilakukan berdasarkan ambang batas jawaban “Ya” (misalnya, $S_{GME} \geq 6$).
4. Skor mentah per butir soal (`rawScores`): skor sebelum pembalikan untuk keperluan audit.
5. Skor terhitung per butir soal (`computedScores`): skor setelah pembalikan dan pembobotan.
6. Skor maksimum yang mungkin (`maxPossibleScore`): dihitung berdasarkan bobot dan nilai opsi tertinggi.

Untuk instrumen GPIUS-2 yang menggunakan metode *Deficient Self-Regulation* (DSR), `computeScore` mengimplementasikan *rollup* orde kedua: skor DSR merupakan agregasi dari dua subskala primer ($S_{DSR} = S_{CP} + S_{CU}$), sedangkan skor total asesmen (`totalScore`) tetap merupakan penjumlahan keseluruhan 15 butir soal ($S_{total} = \sum_{i=1}^{15} v_i$). Penanganan jawaban kosong (*missing-answer*) ditolak sebelum komputasi melalui `validateCompleteness`, sehingga mesin penskoran mengasumsikan data masukan lengkap. Batas rentang interpretasi (*interpretation-boundary inclusivity*) diproses secara inklusif ($min \leq score \leq max$).

Sebagai referensi untuk reproduktibilitas ilmiah, berikut adalah spesifikasi algoritma *scoring engine* (Pseudocode 1):

Algorithm 1 Algoritma Komputasi Skor (computeScore)

Require: *answers* (Map ID Soal $\rightarrow$ Nilai Jawaban), *questions* (Daftar Metadata Soal)

Ensure: *totalScore*, *dimensionScores*, *rawScores*, *computedScores*

```

1:  $total \leftarrow 0, dimScores \leftarrow \{\}$ 
2: for each  $q \in questions$  do
3:    $val \leftarrow answers[q.id]$ 
4:   if  $q.isReversed$  then
5:      $scaleMax \leftarrow \max(q.options), scaleMin \leftarrow \min(q.options)$ 
6:      $val \leftarrow (scaleMax - val) + scaleMin$ 
7:   end if
8:    $finalVal \leftarrow val \times q.weight$ 
9:    $total \leftarrow total + finalVal$ 
10:  if  $q.dimension \neq \text{null}$  then
11:     $dimScores[q.dimension] \leftarrow (dimScores[q.dimension] ?? 0) + finalVal$ 
12:  end if
13: end for
14: if instrument is GPIUS-2 then
15:    $dimScores['DSR'] \leftarrow dimScores['CP'] + dimScores['CU']$ 
16: end if
17: return  $total, dimScores$ 

```

Algoritma pada Pseudocode 1 di atas hanya menghasilkan skor numerik; pemetaan skor tersebut menjadi label yang dapat dipahami pengguna awam dilakukan oleh fungsi terpisah berikut.

4.3.3 Fungsi lookupInterpretation (Lapisan Interpretasi)

Berbeda dengan empat fungsi inti penilaian yang bersifat murni, `lookupInterpretation(testId, totalScore, dimension?)` merupakan fungsi *tidak murni* yang mengkueri tabel `result_interpretations` untuk memetakan skor numerik ke label interpretasi yang dapat dibaca manusia. Fungsi ini mengembalikan label, deskripsi, rekomendasi, dan tingkat keparahan (*severity level*). Pada kasus di mana tidak

ditemukan rentang yang sesuai (*interpretation miss*), fungsi mencatat peringatan ke Sentry tanpa melempar kesalahan, sehingga hasil asesmen tetap tersedia meskipun tanpa interpretasi.

4.3.4 Fungsi Validasi

Dua fungsi validasi, yaitu `validateAnswerValues` dan `validateCompleteness`, memastikan integritas data sebelum proses komputasi skor dimulai. Kedua fungsi bersifat murni (tanpa akses basis data), sehingga dapat diuji secara terisolasi. `validateAnswerValues` memeriksa bahwa setiap nilai jawaban berada dalam rentang opsi yang valid, sedangkan `validateCompleteness` memastikan semua butir soal wajib telah dijawab.

4.3.5 Instrumen yang Diimplementasikan

Empat instrumen psikometri berhasil dikonfigurasi pada platform:

1. **PSS-10** (*Perceived Stress Scale*): 10 butir soal, skala Likert 0–4, dua subskala (ketidakberdayaan dan efikasi diri), dengan 4 butir soal terbalik.
2. **GPIUS-2** (*Generalized Problematic Internet Use Scale 2*): 15 butir soal, skala Likert 1–5, 5 subskala dengan *rollup* DSR orde kedua.
3. **SRS** (*Simplified Resilience Scale*): 11 butir soal, skala Likert 1–6, 3 aspek deskriptif.
4. **SRQ-29** (*Self-Reporting Questionnaire*): 29 butir soal, jawaban biner (Ya/Tidak), 4 domain skrining.

4.4 Implementasi API tRPC

Seluruh 61 prosedur tRPC diimplementasikan dalam 17 *file* prosedur yang diorganisasi ke dalam 16 *router namespace*. Setiap prosedur mendefinisikan skema masukan menggunakan Zod dan dihubungkan ke salah satu dari lima tingkat *middleware* autentikasi.

4.4.1 Komposisi Router

File `src/server/trpc/router.ts` berfungsi sebagai *barrel file* yang mengomposisikan seluruh *router* menjadi satu *app router*. *Router* dikelompokkan berdasarkan domain: *router* publik (`publicTests`, `sessions`, `results`, `health`), *router* pengguna (`profile`), dan *router* admin (`admin`, `adminDashboard`, `adminTests`, `adminQuestions`, `adminScales`, `adminResults`, `reportRequests`, `adminAccounts`, `adminUserAccounts`, `adminAudit`, `adminProfile`).

4.4.2 Structural Lock

Fitur keamanan data yang krusial pada prosedur admin adalah mekanisme *structural lock*. Ketika sebuah instrumen tes telah memiliki sesi asesmen yang tercatat (diperiksa melalui fungsi `getSessionCount`), prosedur `updateTest`, `updateQuestion`, `deleteQuestion`, `reorderQuestions`, dan `updateRange` memblokir perubahan pada kolom yang sensitif terhadap penilaian, seperti `slug`, `scoringMethod`, bobot soal (`weight`), dan nilai opsi (`value`). Mekanisme ini mencegah invalidasi retroaktif terhadap data riset yang telah terkumpul.

4.4.3 Operasi Idempoten

Prosedur `saveProgress` menggunakan pola `ON CONFLICT ... DO UPDATE` (PostgreSQL UPSERT) pada *constraint* unik komposit (`session_id`, `question_id`), sehingga penyimpanan progres bersifat idempoten: pemanggilan berulang dengan data yang sama menghasilkan efek yang identik. Pendekatan ini menangani dengan aman skenario koneksi terputus di mana klien mungkin mengirim ulang jawaban yang sama.

4.5 Implementasi Autentikasi

Sistem autentikasi diimplementasikan sebagai dua subsistem yang sepenuhnya terpisah, masing-masing dengan mekanisme *token*, penyimpanan *cookie*, dan siklus hidup sesi yang independen.

4.5.1 Autentikasi Pengguna Publik (Auth.js v5)

Auth.js (NextAuth v5, versi 5.0.0-beta.30) digunakan untuk autentikasi pengguna publik dengan konfigurasi berikut:

1. **Strategi sesi:** JWT (`session: { strategy: "jwt" }`).
2. **Provider:** `CredentialsProvider` dengan validasi surel dan kata sandi.
3. **Password hashing:** `bcryptjs` dengan *cost factor* 12.
4. **Adaptor:** `CHPDrizzleAdapter` kustom yang dipersiapkan untuk integrasi OAuth di masa mendatang.

5. **Callbacks:** *callback* JWT menanamkan `userId` dan `role` ke dalam *token*; *callback session* meneruskan informasi ini ke konteks sesi.

4.5.2 Autentikasi Administrator (jose JWT)

Subsistem autentikasi admin diimplementasikan secara mandiri menggunakan pustaka `jose` tanpa ketergantungan pada `Auth.js`:

1. **Algoritma:** HS256 dengan kunci rahasia `ADMIN_JWT_SECRET` (minimal 32 karakter).
2. **Sliding window:** setiap permintaan admin yang valid memperpanjang masa berlaku *token* 30 menit dari waktu saat ini.
3. **Batas keras:** *token* tidak dapat diperpanjang melebihi 8 jam dari waktu penerbitan awal (`originalIat`), memaksa *login* ulang untuk sesi yang sangat panjang.
4. **Penyimpanan:** JWT disimpan dalam *cookie* bernama `admin-token`, dikelola melalui modul `cookies.ts` yang menggunakan `ctx.resHeaders` (bukan `cookies()`) untuk kompatibilitas dengan konteks `tRPC`.
5. **Revokasi:** prosedur `logout` menetapkan kolom `sessionInvalidated` `At` di basis data sebelum menghapus *cookie*. *Middleware* `adminProcedure` memeriksa kolom ini pada setiap permintaan, sehingga *token* yang dicuri setelah *logout* tidak dapat digunakan.

4.5.3 *Rate Limiting* pada *Login Admin*

Modul `rate-limiter.ts` mengimplementasikan *rate limiting in-memory* berlapis dua (dengan catatan bahwa status *in-memory* bersifat volatil pada arsitektur *serverless* dan dirancang sebagai pengamanan dasar sebelum implementasi Redis persisten pada iterasi masa depan):

1. **Berbasis IP:** 5 kegagalan dalam jendela waktu 5 menit memicu penurunan (*cooldown*) yang pulih secara otomatis setelah jendela berakhir.
2. **Berbasis akun:** 10 kegagalan kumulatif mengunci akun secara permanen, memerlukan intervensi `super_admin` melalui prosedur `unlockAdmin`. Mekanisme ini disimpan dalam memori (*in-memory*). Meskipun memadai pada lingkungan lokal, pendekatan ini memiliki keterbatasan pada lingkungan *serverless* (seperti Vercel) karena status memori akan diatur ulang (*reset*) setiap kali fungsi instansiasi baru berjalan.

4.6 Implementasi Panel Administrasi

Panel administrasi diakses melalui rute `/admin/(dashboard)/*` dan dilindungi oleh penjaga rute `proxy.ts` yang memeriksa keberadaan *cookie* `admin-token`.

Panel terdiri atas delapan modul utama:

1. ***Dashboard*:** statistik agregat platform: jumlah asesmen selesai, sesi aktif, tingkat penyelesaian, tes terpopuler, dan hasil terbaru.

2. **Manajemen Instrumen** (`adminTests`): CRUD tes dengan transisi status `draft` → `published` → `archived`, dilengkapi *structural lock* untuk instrumen yang telah memiliki sesi.
3. **Manajemen Soal** (`adminQuestions`): penambahan, penyuntingan, penghapusan, dan pengurutan ulang butir soal beserta opsi jawaban.
4. **Konfigurasi Skala** (`adminScales`): pengelolaan rentang interpretasi per dimensi.
5. **Data Hasil** (`adminResults`): tabel data dengan paginasi, filter, dan pengurutan; ekspor data dengan batas 5000 baris yang mencakup jawaban per butir soal.
6. **Permintaan Laporan** (`reportRequests`): alur persetujuan bertahap (*pending* → *reviewed* → *approved/rejected*) dengan dukungan persetujuan *batch*.
7. **Manajemen Akun Admin** (`adminAccounts`, khusus `super_admin`): pembuatan akun, perubahan peran, aktivasi/deaktivasi, pembukaan kunci, dan invalidasi sesi paksa. Dilengkapi pengaman *last super admin* yang mencegah penonaktifan `super_admin` terakhir.
8. **Jejak Audit** (`adminAudit`): log paginasi dengan filter cepat, pembatasan peran, dan resolusi label entitas. Setiap mutasi administratif menulis ke tabel `audit_logs`.

4.7 Implementasi Generasi PDF dan Pengiriman Surel

Generasi laporan hasil asesmen menggunakan `@react-pdf/renderer` versi 4.5.1, yang memungkinkan pembuatan dokumen PDF menggunakan komponen React. Laporan yang dihasilkan mencakup informasi tes, skor total, skor per dimensi, label interpretasi, dan rekomendasi.

Pengiriman surel menggunakan layanan Resend versi 6.12.2, yang menyediakan API transaksional untuk pengiriman surel dari lingkungan *serverless*. Alur lengkap, dari penerimaan permintaan melalui prosedur `approve` pada `router reportRequests`, generasi PDF, hingga pengiriman surel, diorkestrasi secara sekuensial dalam satu prosedur tRPC (bukan atomik berbasis transaksi basis data), dengan mengandalkan penjaga idempotensi untuk mencegah pengiriman berulang.

4.8 Pengujian Unit

Pengujian unit dilaksanakan menggunakan Vitest versi 4.1.5 dengan konfigurasi yang didefinisikan dalam `vitest.config.ts`. Total 213 kasus uji (*test cases*) ditulis dalam 25 *file* uji yang diorganisasi berdasarkan modul. Tabel 4.1 menyajikan inventaris lengkap.

Tabel 4.1: Inventaris *File* Pengujian Unit

<i>File</i> Uji	Jumlah Kasus Uji
<code>data/interpretations.test.ts</code>	40
<code>scoring/engine.test.ts</code>	20
<code>scoring/reverseScore.test.ts</code>	20

<i>File Uji</i>	<i>Jumlah Kasus Uji</i>
scoring/validation.test.ts	19
admin-questions/admin-questio ns.test.ts	15
admin-accounts/admin-accounts. test.ts	10
scoring/lookupInterpretation. test.ts	9
encryption.test.ts	9
admin-scales/admin-scales.tes t.ts	8
data/srq29-questions.test.ts	7
reports/report-requests.test. ts	6
admin-auth/jwt.test.ts	5
admin-tests/admin-tests.test. ts	5
admin-auth/rate-limiter.test. ts	4
admin-profile/admin-profile.t est.ts	4
admin-audit/admin-audit.test. ts	3

<i>File Uji</i>	<i>Jumlah Kasus Uji</i>
admin-results/admin-results-delete.test.ts	3
admin-accounts/admin-user-accounts.test.ts	2
schema/admin-tests-guards.test.ts	2
schema/admin-schema.test.ts	2
keyToOptionValue.test.ts	6
save-progress-guard.test.ts	5
export-utils.test.ts	4
seed-policy.test.ts	4
smoke.test.ts	1
Total	213

Modul *scoring engine* diuji dengan 59 kasus uji yang tersebar dalam tiga *file* (`engine.test.ts`, `reverseScore.test.ts`, `validation.test.ts`), mencakup skenario penjumlahan sumatif, pembalikan skor, pengelompokan dimensional, penanganan masukan kosong, validasi batas opsi, dan verifikasi kelengkapan jawaban.

4.9 Pengujian CI/CD

Pipeline CI/CD diimplementasikan dalam satu *workflow* GitHub Actions (`.github/workflows/ci.yml`) yang diaktifkan pada setiap *push* ke cabang

master dan setiap *pull request* yang menargetkan master. *Pipeline* terdiri atas dua *job* yang berjalan secara paralel:

1. **Job** build-and-test: menggunakan Node.js 22 dan pnpm 9, menjalankan tiga *gate* berurutan:

(a) pnpm exec eslint .: pemeriksaan *linting*.

(b) pnpm tsc -noEmit: pemeriksaan tipe.

(c) pnpm build: *build* produksi Next.js.

2. **Job** test: menggunakan Node.js 22 dan pnpm 9, menjalankan pnpm test untuk mengeksekusi seluruh *test suite* Vitest.

Kedua *job* menggunakan *caching* pnpm untuk mempercepat instalasi dependensi. Sebuah *pull request* tidak dapat di-merge apabila salah satu *job* gagal.

4.10 Deployment dan Opsi Instalasi Mandiri

Platform CHP dirancang agar dapat dioperasikan melalui dua jalur yang menjalankan basis kode yang sama persis: (1) *deployment cloud serverless*, yang menjadi jalur produksi saat ini, dan (2) instalasi mandiri (*on-premise*) pada server milik institusi, baik secara manual maupun melalui kontainer Docker. Seluruh perbedaan lingkungan (lokasi basis data, kunci rahasia, layanan pihak ketiga) ditangani melalui variabel konfigurasi pada berkas `.env`, tanpa perubahan kode. Dokumentasi teknis kedua jalur tersedia pada repositori GitHub publik proyek ini.

4.10.1 *Deployment Cloud (Vercel dan Neon)*

Jalur produksi saat ini menggunakan dua layanan terkelola:

1. **Vercel** untuk komputasi: setiap *push* ke cabang master secara otomatis memicu *deployment* produksi. Setiap *pull request* mendapatkan *preview deployment* dengan URL unik untuk pengujian.
2. **Neon PostgreSQL** untuk basis data: menggunakan *pooled connection string* dengan fitur *scale-to-zero* yang menghentikan komputasi basis data saat tidak ada permintaan, mengurangi biaya operasional.

Seluruh 16 *environment variable* dikonfigurasi melalui *dashboard* Vercel dan tidak disimpan dalam repositori kode sumber. Kunci rahasia (AUTH_SECRET, ADMIN_JWT_SECRET, ENCRYPTION_KEY) dihasilkan menggunakan generator kriptografis yang aman dan disimpan secara terenkripsi oleh Vercel.

Pemantauan kesalahan (*error monitoring*) menggunakan Sentry versi 10.50.0, yang terintegrasi baik pada sisi *server* (`sentry.server.config.ts`) maupun sisi *edge* (`sentry.edge.config.ts`). Setiap kesalahan yang tidak tertangani pada produksi secara otomatis dilaporkan ke *dashboard* Sentry dengan konteks *stack trace* dan metadata permintaan.

4.10.2 *Instalasi Manual (On-Premise)*

Apabila institusi menghendaki sistem berjalan sepenuhnya pada server sendiri, instalasi manual dilakukan melalui langkah-langkah berikut:

1. **Penyiapan prasyarat:** Node.js versi 22 (LTS) atau lebih baru, pnpm versi 9 atau lebih baru, Git, serta sebuah *instance* PostgreSQL 17 (dapat berupa PostgreSQL yang dipasang langsung pada server institusi maupun layanan terkelola).
2. **Pengambilan kode sumber:** `git clone` dari repositori GitHub proyek, dilanjutkan `pnpm install` untuk memasang seluruh dependensi sesuai *lockfile*.
3. **Konfigurasi lingkungan:** menyalin `.env.example` menjadi `.env.local`, lalu mengisi 14 variabel konfigurasi aktif (termasuk `DATABASE_URL` yang diarahkan ke PostgreSQL institusi) beserta dua variabel *bootstrap* akun admin awal (`ADMIN_EMAIL`, `ADMIN_PASSWORD`).
4. **Inisialisasi basis data:** `pnpm db:push` menerapkan seluruh skema (20 tabel dan 7 *enum*) ke basis data tujuan, dilanjutkan `pnpm db:seed` yang bersifat idempoten untuk mengisi empat instrumen psikometri, rentang interpretasi, sitasi, dan akun `super_admin` pertama.
5. **Build dan eksekusi:** `pnpm build` menghasilkan *build* produksi, lalu `pnpm start` menjalankan aplikasi pada *port* 3000 (dapat diubah).
6. **Publikasi:** untuk akses melalui HTTPS, aplikasi ditempatkan di belakang *reverse proxy* (misalnya Nginx) yang menangani sertifikat TLS dan meneruskan permintaan ke *port* aplikasi.

4.10.3 Instalasi Berbasis Kontainer (Docker)

Sebagai alternatif yang lebih terisolasi, repositori menyediakan *Dockerfile* multi-tahap (*multi-stage*) beserta dua berkas Docker Compose. *Dockerfile* tersusun atas empat tahap:

1. *deps*: berbasis citra `node:24-alpine`, memasang dependensi menggunakan `pnpm` dengan *lockfile* beku (*frozen lockfile*) agar versi dependensi identik di mesin mana pun.
2. *dev*: tahap pengembangan yang memetakan (*bind-mount*) direktori proyek ke dalam kontainer, sehingga perubahan kode langsung termuat ulang (*hot reload*).
3. *builder*: menjalankan *build* produksi Next.js di dalam kontainer; kunci rahasia tidak pernah disertakan pada tahap ini.
4. *runner*: citra produksi minimal yang hanya berisi keluaran *standalone* Next.js, dijalankan sebagai pengguna *non-root* demi keamanan, dan mengekspos *port* 3000.

Pengoperasiannya cukup satu perintah: `docker compose -env-file .env.local up -build` untuk mode pengembangan, atau `docker compose -f docker-compose.prod.yml -env-file .env.local up -build` untuk citra produksi teroptimasi. Seluruh konfigurasi tetap bersumber dari berkas `.env.local` yang sama dengan instalasi manual; inisialisasi basis data (langkah 4 pada Subbab 4.10.2) tetap dijalankan satu kali sebelum penggunaan pertama.

4.10.4 Kebutuhan Perangkat untuk Instalasi Mandiri

Tabel 4.2 merinci kebutuhan perangkat keras dan perangkat lunak server untuk instalasi mandiri. Spesifikasi ini tergolong ringan karena aplikasi Next.js *standalone* berukuran kecil dan beban kerja platform lebih bersifat *I/O-bound* (menunggu basis data) daripada *CPU-bound*.

Tabel 4.2: Kebutuhan Perangkat Server untuk Instalasi Mandiri

Komponen	Minimum	Direkomendasikan
CPU	2 <i>core</i> x86-64	4 <i>core</i>
RAM	4 GB	8 GB (aplikasi + PostgreSQL + sistem operasi)
Penyimpanan	20 GB SSD	40 GB SSD
Sistem operasi	Linux <i>server</i> 64-bit (mis. Ubuntu Server 22.04 LTS)	sama; Windows/macOS didukung melalui Docker Desktop untuk kebutuhan pengembangan
Perangkat lunak (jalur Docker)	Docker Engine v24+ dengan Compose v2	sama
Perangkat lunak (jalur manual)	Node.js 22 LTS, pnpm 9+, PostgreSQL 17	sama
Jaringan	Satu <i>port</i> TCP untuk aplikasi (bawaan 3000, dapat diubah)	<i>reverse proxy</i> dengan TLS untuk akses HT-TPS publik

Ringan atau tidaknya kebutuhan pada Tabel 4.2 tersebut turut dipengaruhi oleh sifat kontainer itu sendiri, sebagaimana diuraikan berikut.

4.10.5 Isolasi Kontainer dan Koeksistensi dengan Aplikasi Lain

Pertanyaan yang wajar muncul ketika institusi mempertimbangkan instalasi mandiri adalah: apakah server institusi (beserta aplikasi-aplikasi lain yang sudah berjalan di dalamnya) harus menyesuaikan diri terhadap kebutuhan platform ini? Jawabannya tidak, dan justru inilah alasan utama dukungan Docker disediakan. Sebuah kon-

tainer mengemas aplikasi beserta seluruh lingkungan eksekusinya (versi Node.js, dependensi, konfigurasi) menjadi satu unit yang terisolasi dari sistem operasi induk.

Konsekuensinya:

1. Server tidak perlu memasang Node.js, pnpm, atau dependensi apa pun secara global; semuanya berada di dalam citra kontainer.
2. Aplikasi lain pada server yang sama tidak terpengaruh dan tidak perlu diubah, karena tidak ada dependensi yang dibagi-pakai; konflik versi pustaka antar-aplikasi tidak dapat terjadi.
3. Satu-satunya sumber daya yang berpotensi bersinggungan adalah nomor *port* jaringan, dan pemetaannya sepenuhnya dapat dikonfigurasi (misalnya 8080 : 3000 apabila *port* 3000 telah terpakai aplikasi lain).
4. Basis data pun dapat dijalankan sebagai kontainer PostgreSQL tersendiri, sehingga seluruh tumpukan berdiri sendiri di atas server institusi tanpa ketergantungan eksternal.

Tabel 4.3 membandingkan kedua jalur operasional secara ringkas sebagai dasar pengambilan keputusan institusi.

Karena kedua jalur menjalankan basis kode dan skema basis data yang identik, perpindahan dari satu jalur ke jalur lain tidak memerlukan modifikasi kode, cukup penyesuaian variabel lingkungan dan migrasi isi basis data. Dengan demikian, apabila di kemudian hari UKRIDA mensyaratkan seluruh data berada pada infrastruktur sendiri, jalur instalasi mandiri pada Subbab 4.10.2 dan 4.10.3 dapat ditempuh tanpa menyentuh satu baris kode pun.

Tabel 4.3: Perbandingan Jalur *Deployment Cloud* dan Instalasi Mandiri

Aspek	<i>Cloud</i> (Vercel + Neon)	<i>On-Premise</i> (Docker + PostgreSQL mandiri)
Komputasi	<i>Serverless</i> , skala otomatis	Kapasitas tetap sesuai spesifikasi server
Basis data	Neon PostgreSQL terkelola	PostgreSQL yang dikelola institusi
Lokasi data	Pusat data penyedia <i>cloud</i>	Sepenuhnya di server institusi
Pembaruan sistem	Otomatis pada setiap <i>push</i> ke <i>master</i>	Manual (<i>git pull</i> dan <i>build</i> ulang citra)
TLS dan domain	Dikelola otomatis oleh Vercel	Dikelola institusi melalui <i>reverse proxy</i>
Keamanan infrastruktur	Tanggung jawab penyedia tersertifikasi (SOC 2, ISO 27001)	Tanggung jawab tim TI institusi
Biaya	Langganan layanan (tersedia tingkat gratis)	Perangkat keras, listrik, dan tenaga pengelola
Kesesuaian	Operasional publik dengan pemeliharaan minimal	Kebutuhan kedaulatan data internal institusi

4.11 Pemenuhan Permintaan Dosen Pemilik Proyek

Selama pengembangan Platform Asesmen Digital CHP, dosen pemilik proyek menyampaikan 22 permintaan fungsional yang menjadi acuan pengembangan. Dari 22 permintaan tersebut, 18 permintaan telah tercapai dan 4 permintaan belum tercapai pada akhir periode kerja praktik. Tabel lengkap beserta status dan keterangan setiap butir permintaan disajikan pada Lampiran A.

A. Instrumen Asesmen. Permintaan penambahan dua instrumen baru, yaitu GPIUS-2 (*Generalized Problematic Internet Use Scale 2*) dan SRS (*Simplified Resilience Scale*), berhasil dipenuhi dan ditambahkan ke katalog asesmen bersama instrumen yang telah ada sebelumnya, yaitu PSS-10 dan SRQ-29. Setiap instrumen dikonfigurasi secara lengkap meliputi butir soal, opsi jawaban, pembobotan, dimensi, dan rentang interpretasi, dengan komputasi skor yang diverifikasi melalui 59 kasus

uji pada *scoring engine*.

B. Alur Asesmen Peserta. Alur asesmen peserta dilengkapi dengan visualisasi hasil secara *real-time* yang ditampilkan segera setelah penyelesaian melalui komponen grafik Recharts (*gauge*, *radar*, diagram batang). Formulir demografis menyediakan pemilihan domisili bertingkat dua level (provinsi → kota/kabupaten) menggunakan dataset statis 38 provinsi yang bersumber dari klasifikasi BPS (Badan Pusat Statistik), dilengkapi opsi “Lainnya (ketik manual)” pada pemilihan provinsi maupun kota/kabupaten untuk mengakomodasi wilayah yang tidak terdaftar. Penyimpanan jawaban dilakukan secara otomatis pada setiap perubahan melalui prosedur `saveProgress` dengan pola *upsert* idempoten, memungkinkan peserta melanjutkan sesi yang terputus.

C. Pelaporan dan Ekspor. Pengguna dapat mengajukan permintaan laporan melalui formulir tervalidasi; permintaan tersimpan pada tabel `report_requests` untuk diproses administrator. Sistem menyediakan ekspor data dalam format CSV dan XLSX menggunakan pustaka SheetJS, dengan mekanisme *filtered download* yang memastikan data yang diunduh bersifat dinamis mengikuti filter aktif (jenis kelamin, provinsi, kota, rentang usia, rentang skor, kategori, dan rentang tanggal). Setiap pengguna memiliki laporan komprehensif mencakup visualisasi data pribadi, analisis per-dimensi, dan interpretasi mendetail. Hasil mendalam dapat diunduh dalam format PDF (melalui `@react-pdf/renderer`), XLSX, dan CSV.

D. Dasbor dan Visualisasi Admin. Panel administrasi menyajikan dasbor dengan visualisasi agregat yang mendukung filter berdasarkan jenis instrumen dan rentang tanggal pada dasbor utama, serta filter lanjutan (pencarian nama, jenis kela-

min, provinsi, kota, usia, skor, kategori, dan tanggal) pada halaman data hasil per instrumen. Dasbor antrean permintaan laporan memungkinkan administrator melihat, menyetujui, atau menolak permintaan; persetujuan memicu generasi PDF dan pengiriman surel yang diorkestrasi secara sekuensial. Komponen grafik dilengkapi fitur unduh gambar menggunakan pustaka `html-to-image` (fungsi `toPng`) yang mengkonversi elemen DOM ke format PNG dengan resolusi $2 \times \text{pixel ratio}$. Kolom tabel hasil mendukung pengurutan dinamis *ascending* dan *descending* pada enam parameter (nama, jenis kelamin, usia, skor, kategori, dan tanggal tes) baik pada sisi server melalui kueri Drizzle ORM maupun pada sisi klien. Perlu dicatat bahwa data historis hasil asesmen mungkin memiliki heterogenitas skema (seperti ketiadaan `result_label` atau `dimensionMaxScores`) akibat evolusi struktur basis data pada fase awal pengembangan, yang ditangani secara tangguh oleh antarmuka.

E. Manajemen dan Keamanan. Administrator dapat melakukan operasi CRUD penuh atas tes, butir soal, opsi jawaban, dan rentang interpretasi melalui modul manajemen asesmen yang dilengkapi mekanisme *structural lock*. Setiap modifikasi administratif direkam dalam tabel `audit_logs` mencakup aktor, aksi, entitas, serta nilai sebelum dan sesudah. Sistem menerapkan RBAC empat peran (`super_admin`, `admin`, `psychiatrist`, `researcher`) dengan lima tingkat *middleware* otorisasi: `publicProcedure` tanpa autentikasi, `protectedProcedure` untuk sesi `Auth.js`, `adminProcedure` untuk seluruh peran `admin`, `adminMutationProcedure` yang membatasi operasi tulis kepada `super_admin` dan `admin`, serta `superAdminProcedure` yang eksklusif untuk `super_admin`. Peran `psychiatrist` dan `researcher` memperoleh akses baca saja (*read-only*) terhadap data hasil dan statistik. Modul *Account*

Management memungkinkan *Super Admin* untuk membuat, memperbarui, dan mengatur hak akses akun administrator.

4.11.1 Permintaan yang Belum Tercapai

Empat permintaan belum tercapai pada akhir periode kerja praktik. Pertama, peringatan otomatis pada pertanyaan yang belum terisi (butir 19) belum diimplementasikan karena kompleksitas sinkronisasi status pengisian antara penyimpanan sisi-klien (`localStorage`) dan persistensi sisi-server secara simultan; sebagai mitigasi, tombol “Kirim Asesmen” dinonaktifkan secara *default* dan hanya aktif setelah seluruh pertanyaan terjawab, sehingga pengiriman yang tidak lengkap tetap tercegah. Kedua, fitur *import* data massal dalam format XLS/XLSX (butir 20) belum tercapai karena kompleksitas validasi skema terhadap data historis yang beragam format dan struktur, sehingga ditangguhkan ke iterasi berikutnya demi menjaga integritas data penelitian. Ketiga dan keempat, fitur *Contact Us* untuk mengirim pesan ke surel administrator (butir 21) dan fitur *Direct Mail Responder* untuk membalas pesan dari dasbor (butir 22) belum tercapai karena memerlukan integrasi layanan surel dua arah yang melampaui cakupan waktu kerja praktik.

Keempat permintaan yang belum tercapai telah diidentifikasi sebagai pekerjaan lanjutan dan didokumentasikan pada Subbab 5.2 sebagai saran pengembangan di masa mendatang.

Tabel pemenuhan selengkapannya yang mencakup seluruh 22 permintaan beserta status dan keterangan disajikan pada Lampiran A.

BAB 5

PENUTUP

5.1 Kesimpulan

Berdasarkan pelaksanaan Kerja Praktik yang telah dilakukan, dapat ditarik kesimpulan sebagai berikut:

1. Platform Asesmen Psikologi Digital untuk *Center for Health Psychology* (CHP) UKRIDA berhasil dirancang dan diimplementasikan sebagai aplikasi web *full-stack* menggunakan Next.js 16, TypeScript 6.0.3 (`strict: true`), tRPC v11, dan Drizzle ORM di atas Neon PostgreSQL *serverless*. Platform melayani empat aktor pengguna utama, yaitu peserta anonim, pengguna terdaftar, administrator, dan super admin, melalui 61 prosedur API yang diorganisasi dalam 16 *router namespace*. Selain jalur *cloud* tersebut, platform juga dapat diinstalasi secara mandiri (*on-premise*) pada server institusi, baik manual maupun melalui kontainer Docker, tanpa perubahan kode (Subbab 4.10).
2. Skema basis data yang terdiri atas 20 tabel dan 7 *enum* PostgreSQL berhasil mengakomodasi empat instrumen psikometri dengan karakteristik yang beragam: PSS-10 (dua subskala, skala Likert 0–4), GPIUS-2 (5 subskala, skala Likert 1–5, *rollup* DSR), SRS (3 aspek deskriptif, skala Likert 1–6), dan SRQ-29 (4 kluster, jawaban biner). Pemisahan tabel `answers` dari `results` memungkinkan reproduktibilitas ilmiah melalui

re-scoring.

3. *Scoring engine* yang terdiri atas 4 fungsi murni (inti penilaian) dan 1 fungsi lapisan interpretasi (mengakses basis data) berhasil diimplementasikan dan diverifikasi melalui 68 kasus uji (59 pada inti murni, 9 pada lapisan interpretasi) yang mencakup skenario penjumlahan sumatif, pembalikan skor, pengelompokan dimensional, validasi kelengkapan jawaban, dan pemetaan skor ke label. Seluruh komputasi pada inti penilaian bersifat deterministik dan bebas efek samping.
4. Sistem autentikasi ganda berhasil diimplementasikan: Auth.js v5 untuk pengguna publik dan JWT mandiri melalui jose untuk administrator, dengan lima tingkat *middleware* kontrol akses (publicProcedure, protectedProcedure, adminProcedure, adminMutationProcedure, superAdminProcedure) dan empat peran admin (super_admin, admin, psychiatrist, researcher).
5. Panel administrasi dengan delapan modul (termasuk *dashboard*, manajemen instrumen, konfigurasi skala, data hasil, permintaan laporan, manajemen akun, dan jejak audit) berhasil diimplementasikan, memungkinkan tim psikologi mengelola instrumen dan mengeksport data riset secara mandiri tanpa intervensi pengembang.
6. Keamanan dan privasi data diimplementasikan sesuai prinsip UU PDP melalui strategi perlindungan bertingkat yang mengikuti fungsi data: *hashing* satu arah untuk alamat IP dan *user agent*, enkripsi surel AES-

256-GCM, serta kontrol akses berbasis peran, jejak audit administratif, dan pembekuan pasca-sesi untuk data demografis yang dikonsumsi fungsional, dilengkapi sesi anonim, persetujuan eksplisit, dan *rate limiting* berlapis. Batasan pendekatan ini dinyatakan secara eksplisit pada Subbab 1.5.

7. Pengujian pada level unit dan prosedur dengan total 213 kasus uji dalam 25 *file* uji dan *pipeline* CI/CD dua *job* paralel berhasil menjaga kualitas kode sepanjang siklus pengembangan. Pengujian *end-to-end* otomatis dan uji beban/performa merupakan pekerjaan lanjutan.
8. Seluruh enam tujuan KP yang dirumuskan pada Tabel 1.2 berhasil dicapai dalam batas waktu yang ditentukan. Arsitektur yang dibangun telah dirancang untuk memenuhi target minimum kinerja, meskipun pengukuran metrik secara formal merupakan lingkup pekerjaan lanjutan.

5.2 Saran

Untuk pengembangan Platform CHP di masa mendatang, penulis menyarankan beberapa pekerjaan lanjutan yang berada di luar ruang lingkup KP ini namun akan meningkatkan kualitas dan fungsionalitas platform:

1. **Pengujian *end-to-end* (E2E) otomatis:** Meskipun Playwright telah dikonfigurasi dalam proyek (@playwright/test v1.59.1) dan struktur *file* E2E telah disiapkan di direktori *e2e/*, *pipeline* CI/CD saat ini belum menjalankan *job* E2E secara otomatis. Disarankan untuk menambahkan

job ketiga pada *workflow* GitHub Actions yang menjalankan skenario E2E pada setiap *pull request*.

2. **Pengukuran *Lighthouse* otomatis:** Skor *Lighthouse* yang diklaim pada Tabel 1.2 belum diverifikasi melalui pengukuran otomatis dalam *pipeline* CI/CD. Disarankan untuk mengintegrasikan *Lighthouse CI* ke dalam *workflow* GitHub Actions agar setiap *pull request* menyertakan laporan performa dan aksesibilitas yang terukur.
3. **Integrasi Google OAuth:** Variabel lingkungan `GOOGLE_CLIENT_ID` dan `GOOGLE_CLIENT_SECRET` telah disiapkan dalam `.env.example` namun belum terhubung ke *provider* OAuth pada konfigurasi `Auth.js`. Integrasi ini akan memungkinkan mahasiswa UKRIDA untuk *login* menggunakan akun Google institusional mereka.
4. **Alur pemulihan kata sandi (*password reset*):** Platform saat ini belum menyediakan mekanisme bagi pengguna yang lupa kata sandi. Disarankan untuk mengimplementasikan alur pemulihan melalui surel menggunakan *token* verifikasi sekali pakai yang sudah didukung oleh tabel `verification_tokens`.
5. **Alur verifikasi surel:** Registrasi pengguna saat ini tidak mewajibkan verifikasi surel. Implementasi verifikasi akan meningkatkan keamanan dan memastikan keabsahan surel yang terdaftar.
6. **Antarmuka migrasi sesi tamu ke akun:** Meskipun prosedur `claimSession` telah diimplementasikan di sisi *backend*, antarmuka

pengguna untuk memandu peserta anonim dalam mengklaim sesi mereka ke akun baru perlu dirancang dan diimplementasikan.

7. **Ambang batas cakupan kode dalam CI:** *Pipeline* CI saat ini tidak menerapkan ambang batas minimum cakupan kode. Disarankan untuk mengonfigurasi Vitest agar *build* gagal apabila cakupan kode pada modul *scoring engine* turun di bawah 95%.
8. **Uji coba instalasi mandiri pada server UKRIDA:** Panduan instalasi manual dan berbasis Docker beserta kebutuhan perangkatnya telah didokumentasikan pada Subbab 4.10. Disarankan untuk melaksanakan proyek percontohan (*pilot*) instalasi pada server institusi guna memverifikasi panduan tersebut terhadap lingkungan nyata dan mengukur kebutuhan sumber daya aktual di bawah beban penggunaan sesungguhnya.
9. **Pseudonimisasi data demografis:** Kolom nama pada tabel `session_demographics` saat ini disimpan terbaca karena dibutuhkan secara fungsional (Subbab 3.4). Apabila kebutuhan riset di masa mendatang tidak lagi menuntut identifikasi langsung, disarankan mengganti nama dengan pengenal pseudonim dan menyimpan tabel pemetaannya secara terpisah dengan kontrol akses yang lebih ketat, atau mengevaluasi enkripsi per kolom apabila kemampuan pencarian atas nama bersedia dikorbankan.

DAFTAR PUSTAKA

- Beusenbergh, Maria, dan John Orley. *A User's Guide to the Self-Reporting Questionnaire (SRQ)*. Geneva: World Health Organization, 1994.
- Bhat, Amir Hussain, dan Syed Imtiyaz Hassan Khan. "White Box Testing: A Comprehensive Study on Techniques, Tools, and Effectiveness." *International Journal of Advanced Computer Science and Applications* 12, no. 7 (2021): 310–318.
- Bogner, Justus, dan Manuel Merkel. "To Type or Not to Type?: A Systematic Comparison of the Software Quality of JavaScript and TypeScript Applications on GitHub." Dalam *Proceedings of the 19th International Conference on Mining Software Repositories*, 658–669. New York: ACM, 2022. <https://doi.org/10.1145/3524842.3528454>.
- Caplan, Scott E. "Theory and Measurement of Generalized Problematic Internet Use: A Two-Step Approach." *Computers in Human Behavior* 26, no. 5 (2010): 1089–1097. <https://doi.org/10.1016/j.chb.2010.03.012>.
- Capote, Javier A. "A Comparative Analysis of White-Box and Black-Box Testing Methodologies in Modern Software Development." *Journal of Software Engineering and Applications* 16, no. 4 (2023): 102–119.
- Cohen, Sheldon, Tom Kamarck, dan Robin Mermelstein. "A Global Measure of Perceived Stress." *Journal of Health and Social Behavior* 24, no. 4 (Desember 1983): 385–396. <https://doi.org/10.2307/2136404>.
- Cohn, Mike. *Succeeding with Agile: Software Development Using Scrum*. Upper Saddle River, NJ: Addison-Wesley Professional, 2009.
- Desai, Prathamesh, Rajinder Singh, dan Varad Amilkanthwar. "Optimizing Software Testing Strategies: A Systematic Review of the Test Pyramid Approach." *International Journal of Software Engineering and Testing* 8, no. 1 (2025): 45–62.
- Diener, Ed, Robert A. Emmons, Randy J. Larsen, dan Sharon Griffin. "The Satisfaction With Life Scale." *Journal of Personality Assessment* 49, no. 1 (1985): 71–75.
- Docker Inc. "Docker Documentation." Diakses 24 Juli 2026. <https://docs.docker.com/>.
- Drizzle Team. "Drizzle ORM Documentation." Diakses 1 April 2026. <https://orm.drizzle.team/docs/overview>.
- "From Logic to Toolchains: An Empirical Study of Bugs in the TypeScript Ecosystem." arXiv, 2026. <https://arxiv.org/html/2601.21186v1>.
- Hakim, Arif Rahman, Lisa Mora, Christine H. Leometa, dan Citra Puspa Dimala. "Psychometric Properties of the Perceived Stress Scale (PSS-10) in Indonesian

- Version.” *Jurnal Pengukuran Psikologi dan Pendidikan Indonesia* 13, no. 2 (2024): 117–129. <https://doi.org/10.15408/jp3i.v13i2.35482>.
- Himpunan Psikologi Indonesia. *Kode Etik Psikologi Indonesia*. Jakarta: HIMPSI, 2010.
- Idaiani, Sri, Rofingatul Mubasyiroh, Indri Yunita Suryaputri, Lely Indrawati, dan Ika Dharmayanti. “The Validity of the Self-Reporting Questionnaire-20 for Symptoms of Depression: A Sub-Analysis of the National Health Survey in Indonesia.” *Open Access Macedonian Journal of Medical Sciences* 7, no. E (2019): 1676–1682. <https://doi.org/10.3889/oamjms.2019.9999>.
- Jones, Michael B., John Bradley, dan Nalini Sakimura. “JSON Web Token (JWT).” RFC 7519. Internet Engineering Task Force, Mei 2015. <https://datatracker.ietf.org/doc/html/rfc7519>.
- Khan, Muhammad Ehsan, dan Farmeena Khan. “A Comparative Study of Black Box Testing Techniques.” *ACM Computing Surveys* 55, no. 3 (2022): 1–36.
- Liu, Meihua, Chih-Hung Chen, dan Wei Sun. “Automated Scoring Validation in Computer-Based Psychological Assessment: A Systematic Approach.” *Computers in Human Behavior* 61 (2016): 544–553.
- Manning, Lydia K., Dawn C. Carr, dan Ben Lennox Kail. “Do Higher Levels of Resilience Buffer the Deleterious Impact of Chronic Illness on Disability in Later Life?” *The Gerontologist* 56, no. 3 (2016): 514–524. <https://doi.org/10.1093/geront/gnu068>.
- Neon Inc. “Security Overview.” Diakses 24 Juli 2026. <https://neon.com/docs/security/security-overview>.
- OWASP Foundation. “OWASP Top Ten.” Diakses 1 April 2026. <https://owasp.org/www-project-top-ten/>.
- Pearlin, Leonard I., dan Carmi Schooler. “The Structure of Coping.” *Journal of Health and Social Behavior* 19, no. 1 (1978): 2–21.
- Peng, Sida, Eirini Kalliamvakou, Peter Cihon, dan Mert Demirer. “The Impact of AI on Developer Productivity: Evidence from GitHub Copilot.” arXiv, 2023. <https://arxiv.org/abs/2302.06590>.
- Pieh, Christoph, Sanja Budimir, Elke Humer, dan Thomas Probst. “Comparing Mental Health During the COVID-19 Lockdown and 6 Months After the Lockdown in Austria: A Longitudinal Study.” *Frontiers in Psychiatry* 12 (2021): 625973. <https://doi.org/10.3389/fpsy.2021.625973>.
- PostgreSQL Global Development Group. “PostgreSQL 17 Documentation.” Diakses 1 April 2026. <https://www.postgresql.org/docs/17/>.
- Rebong, Theresia Grace D., Jason Hansel Hambali, dan Ferry Timothy. “Implementasi UU PDP Indonesia Dalam Pengembangan Sistem Kecerdasan Buatan: Tantangan Dan Peluang.” *Cerdika: Jurnal Ilmiah Indonesia* 5, no. 10 (Oktober 2025): 2274–2279. <https://doi.org/10.59141/cerdika.v5i10.2856>.

- Reynaldo, R., dan Yasinta Astin Sokang. "Mahasiswa dan Internet: Dua Sisi Mata Uang? Problematic Internet Use pada Mahasiswa." *Jurnal Psikologi* 43, no. 2 (2016): 107–120. <https://doi.org/10.22146/jpsi.17276>.
- Schwaber, Ken, dan Jeff Sutherland. *The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game*. Scrum.org, 2020. <https://scrumguides.org/scrum-guide.html>.
- Sommerville, Ian. *Software Engineering*. 10th ed. Boston: Pearson, 2016.
- tRPC Team. "tRPC v11 Documentation." Diakses 1 April 2026. <https://trpc.io/docs/>.
- Vercel. "Next.js 16 Documentation." Diakses 1 April 2026. <https://nextjs.org/docs>.
- Wagnild, Gail M., dan Heather M. Young. "Development and Psychometric Evaluation of the Resilience Scale." *Journal of Nursing Measurement* 1, no. 2 (1993): 165–178.